

LAPORAN TEAM BASED PROJECT

Algoritma dan Struktur Data

Topik 6: Smart Library Management & Recommendation System

Dosen Pengampu:

Dr. Eng. Aji Ery Burhandenny, S.T., M.AIT.



Disusun oleh:

Danar Hafidz A'ray	(25051030112)
Anargya Keano Arizona Marici	(25051030096)
Itqon El Fikry Fausta	(25051030083)
Fauzi Fajar Wijaya	(25051030095)
Muhammad Daffaa Adhirajasa	(25051030086)

PRODI TEKNIK ELEKTRO

DEPARTMENT PENDIDIKAN TEKNIK ELEKTRO

FAKULTAS TEKNIK

UNIVERSITAS NEGERI YOGYAKARTA

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, tugas besar proyek yang berjudul “Smart Library Management & Recommendation System” ini dapat diselesaikan dengan baik. Proyek ini disusun sebagai salah satu bentuk penerapan materi Algoritma dan Struktur Data ke dalam sebuah sistem manajemen perpustakaan berbasis Python yang terintegrasi dengan fitur rekomendasi buku.

Dalam proses pengembangan proyek ini, berbagai konsep struktur data dan algoritma diterapkan secara langsung, seperti penggunaan Binary Search Tree (BST) untuk pengelolaan katalog buku, Queue untuk sistem antrean pemesanan, Stack untuk riwayat transaksi dan fitur undo, Graph untuk rekomendasi buku ko-pinjam menggunakan BFS, serta implementasi Shell Sort dan Merge Sort untuk pengolahan laporan data perpustakaan. Selain itu, proyek ini juga dilengkapi dengan pengujian menggunakan pytest serta eksperimen benchmark runtime untuk menganalisis performa struktur data yang digunakan.

Penyusunan proyek ini memberikan banyak pengalaman dan pembelajaran, khususnya dalam perancangan sistem berbasis struktur data, pengelolaan modular program Python, debugging, pengujian unit, serta analisis kompleksitas algoritma. Selama proses pengerjaan, berbagai kendala teknis juga dihadapi, mulai dari pengelolaan pointer pada linked list, integrasi antar modul, hingga validasi hasil sorting dan pengujian otomatis menggunakan pytest.

Kami menyadari bahwa proyek ini masih memiliki kekurangan baik dari sisi implementasi maupun dokumentasi. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi pengembangan sistem yang lebih baik di masa mendatang.

Akhir kata, kami berharap tugas besar proyek ini dapat memberikan manfaat, menambah wawasan dalam bidang algoritma dan struktur data, serta menjadi referensi pengembangan sistem perpustakaan berbasis teknologi di masa depan.

Yogyakarta, Mei 2026

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I	6
1.1. LATAR BELAKANG	6
1.2. TUJUAN SISTEM	6
1.3. BATAS IMPLEMENTASI	6
1.4. DIAGRAM ARSITEKTUR SISTEM	7
BAB II	8
PERANCANGAN SISTEM	8
2.1. Pemilihan Struktur Data Dan Justifikasi	8
2.2. Diagram Interaksi Dalam Modul	9
2.3. Pseudocode Algoritma Utama	9
BAB III	11
IMPLEMENTASI	11
3.1. TAMPILAN AWAL CLI	11
3.2. PENJELASAN KODE PER MODUL	11
3.2.1. Queue — Antrian Pemesanan Buku (modul_1.py)	11
3.2.1.1. CONTOH INPUT/INPUT CLI	12
3.2.2. Stack — Riwayat Transaksi & UNDO (modul_2.py)	12
3.2.2.1. CONTOH INPUT/OUTPUT CLI	13
3.2.3. BST — Katalog Buku (modul_3.py)	13
3.2.3.1. CONTOH INPUT/OUTPUT CLI	13
3.2.4. Graph — Rekomendasi Ko-pinjam (modul_4.py)	14
3.2.4.1. CONTOH INPUT/OUTPUT CLI	14
3.2.5. Sorting — Laporan Bulanan (modul_5.py)	14
3.2.5.1. CONTOH INPUT/OUTPUT CLI	15

3.2.6. CLI — Loop Utama (modul_6.py)	15
3.2.6.1. CONTOH INPUT/OUTPUT CLI	16
BAB IV	17
ANALISIS BIG-O	17
4.1. ANALISIS KOMPLEKSITAS RUANG DAN WAKTU SETIAP FUNGSI UTAMA	17
4.1.1. Queue — Antrian Pemesanan Buku	18
4.1.1.1. CODE FUNGSI UTAMA	19
4.1.1.2. TABEL ANALISIS KOMPLEKSITAS RUANG DAN WAKTU	20
4.1.2. Stack — Riwayat Transaksi & UNDO	20
4.1.2.1. CODE FUNGSI UTAMA	20
4.1.2.2. TABEL ANALISIS KOMPLEKSITAS RUANG DAN WAKTU	21
4.1.3. BST — Katalog Buku	21
4.1.3.1. CODE FUNGSI UTAMA	21
4.1.3.2. TABEL ANALISIS KOMPLEKSITAS RUANG DAN WAKTU	22
4.1.4. Graph — Rekomendasi Ko-pinjam	22
4.1.4.1. CODE FUNGSI UTAMA	22
4.1.4.2. TABEL ANALISIS KOMPLEKSITAS RUANG DAN WAKTU	24
4.1.5. Shell Sort & Merge Sort pada Linked List	24
4.1.5.1. CODE FUNGSI UTAMA	24
4.1.5.2. TABEL ANALISIS KOMPLEKSITAS RUANG DAN WAKTU	25
4.2. TABEL RINGKASAN	25
4.3. PERBANDINGAN TEORITIS VS EMPIRIS	26
BAB V	27
HASIL EKSPERIMEN	27
5.1. SETUP EKSPERIMEN	27
5.2. TABEL RUNTIME 3+ UKURAN DATA	27
5.2.1. Hasil Benchmark Queue & Stack	27

5.2.2. Hasil Benchmark BST	28
5.2.3. Hasil Benchmark Graph BFS	28
5.2.4. Hasil Benchmark Sorting pada Linked List	29
5.2.5. Hasil Benchmark Integrasi 300 Event	30
5.2.6. Perbandingan Shell Sort vs Merge Sort — Visualisasi Tren	30
5.4. RINGKASAN DISKUSI KESESUAIAN	31
BAB VI	31
JAWABAN PERTANYAAN ANALISIS	31
6.1. Pertanyaan 1	31
6.2. Pertanyaan 2	33
6.3. Pertanyaan 3	34
6.4. Pertanyaan 4	35
6.5. Pertanyaan 5	36
BAB VII	38
KESIMPULAN	38
7.1. RINGKASAN PENCAPAIAN.....	38
7.2. REFLEKSI TRADE-OFF DESAIN	38
7.3. SARAN PENGEMBANGAN	39
LAMP.....	40
LAMPIRAN	40
LAMP.1. PEMBAGIAN TUGAS ANGGOTA.....	40
LAMP.2. TABEL RUNTIME LENGKAP	41
LAMP.3. LOG AI	41

BAB I

PENDAHULUAN

1.1. LATAR BELAKANG

Perpustakaan universitas modern seperti UNY Library melayani ribuan anggota setiap harinya. Proses peminjaman, pengembalian, dan pemesanan buku yang tinggi volumenya membutuhkan sistem yang tidak hanya fungsional, tetapi juga efisien secara algoritmis. Ketika sebuah buku populer sedang dipinjam, anggota lain membutuhkan mekanisme antrian yang adil dan transparan. Ketika petugas melakukan kesalahan pencatatan, sistem perlu menyediakan fitur pembatalan transaksi secara instan. Dan untuk meningkatkan pemanfaatan koleksi, sistem idealnya mampu merekomendasikan buku lain yang relevan berdasarkan pola peminjaman anggota.

Proyek ini membangun Smart Library Management & Recommendation System, sebuah sistem perpustakaan digital berbasis Command Line Interface (CLI) yang mengintegrasikan empat struktur data utama: Queue untuk antrian pemesanan, Stack untuk riwayat transaksi dan fitur undo, Binary Search Tree (BST) untuk katalog buku, serta Graf untuk rekomendasi berbasis ko-pinjam. Seluruh struktur data diimplementasikan dari nol tanpa menggunakan library bawaan Python sesuai ketentuan mata kuliah.

1.2. TUJUAN SISTEM

Tujuan dari pengembangan sistem ini adalah:

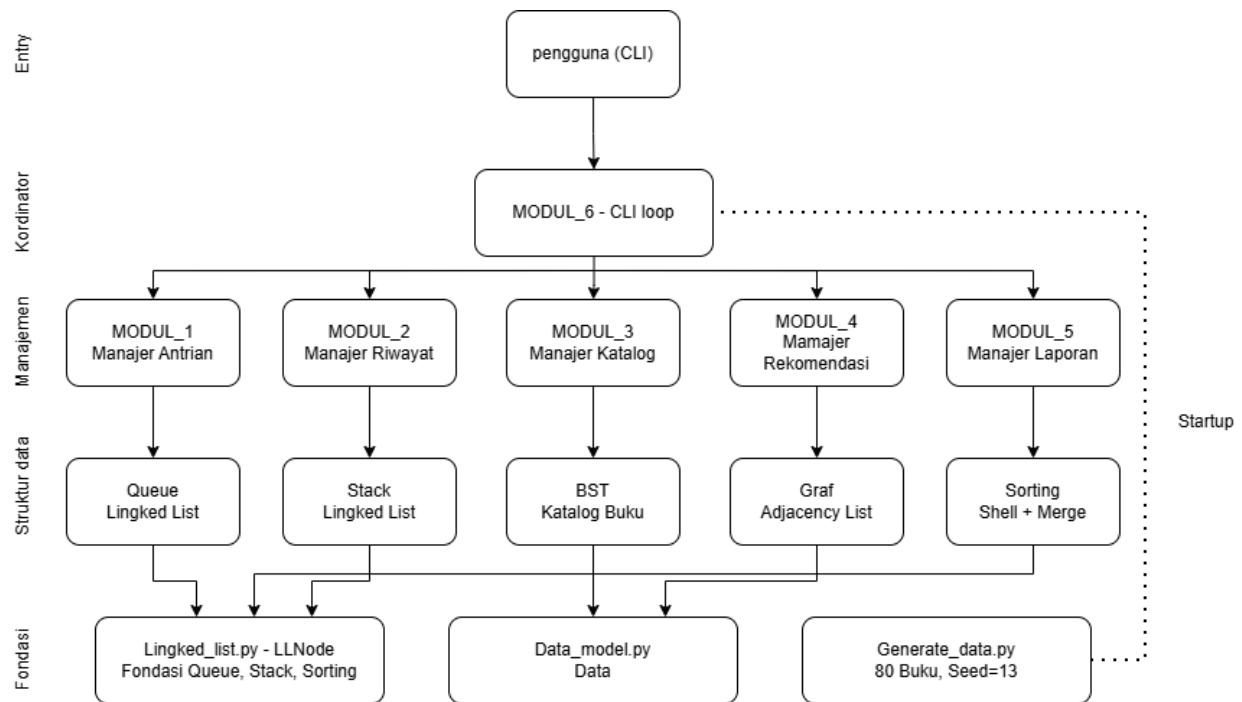
- Mengimplementasikan Queue berbasis Linked List untuk mengelola antrian pemesanan buku per-ISBN secara FIFO dengan operasi $O(1)$.
- Mengimplementasikan Stack berbasis Linked List untuk merekam riwayat transaksi global dan mendukung fitur undo (BATALKAN_TERAKHIR) dengan operasi $O(1)$.
- Mengimplementasikan BST dengan kunci ISBN untuk operasi pencarian, insert, dan update status buku dalam waktu rata-rata $O(\log n)$.
- Mengimplementasikan Graf tak-berarah berbobot dengan BFS untuk sistem rekomendasi buku berdasarkan pola ko-pinjam anggota dalam $O(V+E)$.
- Mengimplementasikan Shell Sort dan Merge Sort pada Linked List untuk menyusun laporan bulanan berurutan berdasarkan durasi dan frekuensi peminjaman.
- Membangun antarmuka CLI interaktif yang menyatukan seluruh fitur dalam satu sistem terpadu dengan seed reproduibel (seed=13).

1.3. BATAS IMPLEMENTASI

- Bahasa pemrograman: Python 3.10 atau lebih baru

- Semua struktur data (Queue, Stack, BST, Graf) diimplementasikan dari nol tanpa menggunakan library bawaan Python seperti collections.deque atau heapq.
- Koleksi: 80 judul buku (ISBN-0001 s.d. ISBN-0080), 60 anggota (format NIM), random seed = 13 (tidak boleh diubah agar hasil program reproduibel)
- Durasi peminjaman default: 14 hari; operasi CLI minimum: 300 event campuran
- Antarmuka sistem: CLI interaktif — setiap perintah menampilkan Big-O operasinya
- Eksperimen benchmark dilakukan untuk $N = 20, 80$, dan 300 sebagai ukuran data representatif.

1.4. DIAGRAM ARSITEKTUR SISTEM



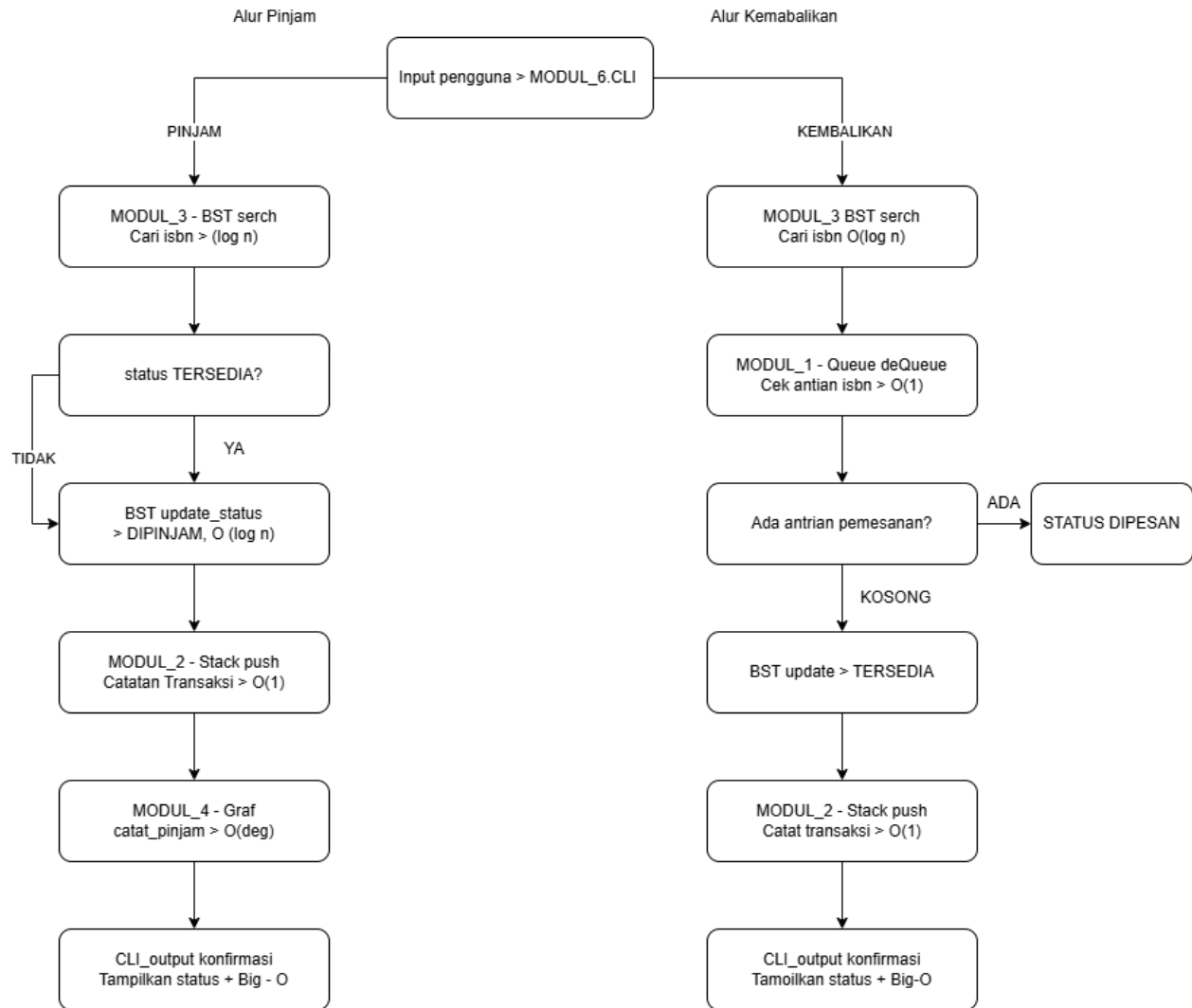
BAB II

PERANCANGAN SISTEM

2.1. Pemilihan Struktur Data Dan Justifikasi

No.	Modul	Struktur Data	Justifikasi Pemilihan	Big-O Utama
1	modul_1 — Antrian Pemesanan	Queue (Linked List)	FIFO mutlak: anggota pertama memesan buku harus dilayani pertama. Linked List memastikan enqueue $O(1)$ dengan pointer tail eksplisit tanpa traversal.	enqueue/dequeue: $O(1)$
2	modul_2 — Riwayat & Undo	Stack (Linked List)	LIFO: transaksi terbaru yang di-undo. Push/pop $O(1)$ karena operasi hanya menyentuh top node tanpa traversal.	push/pop: $O(1)$
3	modul_3 — Katalog Buku	BST (kunci ISBN)	Pencarian $O(\log n)$ rata-rata pada 80 buku. ISBN terurut leksikografis cocok sebagai kunci BST. Mendukung insert, search, delete, dan update_status.	search/insert/delete: $O(\log n)$ avg
4	modul_4 — Rekomendasi	Graf tak- berarah berbobot + BFS	Edge memodelkan frekuensi ko-pinjam. BFS menemukan buku rekomendasi dalam radius ≤ 2 hop secara efisien $O(V+E)$.	BFS: $O(V+E)$
5	modul_5 — Laporan Bulanan	Linked List + Shell Sort + Merge Sort	Shell Sort $\sim O(n^{1.5})$ untuk durasi peminjaman; Merge Sort $O(n \log n)$ untuk frekuensi per ISBN. Keduanya diimplementasikan langsung pada Linked List.	Shell: $\sim O(n^{1.5})$, Merge: $O(n \log n)$

2.2. Diagram Interaksi Dalam Modul



2.3. Pseudocode Algoritma Utama

BST Insert dan Search (modul_3 / bst.py)

```

1. FUNGSI bst_insert(root, buku):
2.   JIKA root == None:
3.     KEMBALIKAN BSTNode(buku)
4.   JIKA buku.isbn < root.buku.isbn:
5.     root.kiri = bst_insert(root.kiri, buku)
6.   JIKA buku.isbn > root.buku.isbn:
7.     root.kanan = bst_insert(root.kanan, buku)
8.   LAIN: root.buku = buku // update duplikat
9.   KEMBALIKAN root
10. // Big-O: O(log n) rata-rata | O(n) worst-case (data terurut)
11.

```

Queue Enqueue/Dequeue (modul_1 / queue.py)

```
1. FUNGSI enqueue(data):
2.   node_baru = LLNode(data)
3.   JIKA tail == None:
4.       head = tail = node_baru // antrian kosong
5.   LAIN:
6.       tail.next = node_baru
7.       tail = node_baru      // geser tail
8.   size += 1
9. // Big-O: O(1) — pointer tail eksplisit, tanpa traversal
10.
11.
```

BFS Rekomendasi (modul_4 / graph.py)

```
1. FUNGSI bfs_rekomendasi(isbn_sumber, max_hop=2):
2.   antrian.enqueue((isbn_sumber, hop=0, bobot=0))
3.   dikunjungi = {isbn_sumber}
4.   SELAMA antrian tidak kosong:
5.       (isbn, hop, bobot) = antrian.dequeue()
6.       UNTUK setiap (tetangga, w) dalam adj[isbn]:
7.           JIKA tetangga tidak dalam dikunjungi:
8.               dikunjungi.tambah(tetangga)
9.               hasil.tambah((tetangga, bobot+w, hop+1))
10.            JIKA hop+1 < max_hop:
11.                antrian.enqueue((tetangga, hop+1, bobot+w))
12.   KEMBALIKAN insertion_sort_turun(hasil)
13. // Big-O: O(V+E)
14.
```

Merge Sort pada Linked List (sorting.py)

```
1. FUNGSI merge_sort_frekuensi(head):
2.   JIKA head == None ATAU head.next == None:
3.       KEMBALIKAN head // basis rekursi
4.   kiri, kanan = split_tengah(head) // slow/fast pointer O(n)
5.   kiri = merge_sort_frekuensi(kiri)
6.   kanan = merge_sort_frekuensi(kanan)
7.   KEMBALIKAN merge_descending(kiri, kanan)
8. // Big-O: O(n log n) | Ruang: O(log n) call stack
9.
```

BAB III

IMPLEMENTASI

3.1. TAMPILAN AWAL CLI

```
1. [SISTEM] Memulai Smart Library System...
2. [SISTEM] 80 buku berhasil di-generate (seed=13).
3. [SISTEM] Katalog BST siap. {'jumlah_buku': 80, 'tinggi': 80}
4. [SISTEM] Antrian per ISBN siap (80 slot).
5.
6.
7. Smart Library Management & Recommendation System
8. ELT60213 Algoritma dan Struktur Data
9. Ketik BANTUAN untuk daftar perintah
10.
11.
12. >> BANTUAN
13.
14.
15. DAFTAR PERINTAH
```

DAFTAR PERINTAH	Big-O
CARI_BUKU <isbn>	$O(\log n)$
PINJAM <nim> <isbn>	$O(\log n)$
KEMBALIKAN <isbn>	$O(\log n)$
PESAN <nim> <isbn>	$O(1)$ enqueue
BATALKAN_PESAN <nim> <isbn>	$O(k)$ rebuild
BATALKAN_TERAKHIR	$O(1)$ pop
REKOMENDASI <isbn>	$O(V+E)$ BFS
ANTRIAN <isbn>	$O(k)$ traversal
KATALOG	$O(n)$ inorder
LAPORAN_BULAN	$O(n^{1.5})/O(n \log n)$
BANTUAN	$O(1)$
KELUAR	-

```
28.
29.
30.
```

3.2. PENJELASAN KODE PER MODUL

3.2.1. Queue — Antrian Pemesanan Buku (modul_1.py)

Queue diimplementasikan berbasis Linked List dengan dua pointer: `_head` (ujung depan, tempat dequeue) dan `_tail` (ujung belakang, tempat enqueue). Pointer `_tail` dipertahankan agar enqueue tidak perlu traversal ke akhir list sehingga tetap $O(1)$. Setiap ISBN memiliki satu Queue sendiri yang disimpan dalam dictionary `{isbn: Queue}` di `ManajerAntrian`.

Operasi `batalkan_pesanan` tidak bisa dilakukan secara langsung pada Queue karena tidak mendukung random delete. Solusinya adalah rebuild: semua elemen diambil ke list sementara $O(k)$, NIM yang dibatalkan difilter, lalu Queue baru dibangun tanpa NIM tersebut. Big-O keseluruhan: $O(k)$ dengan k = panjang antrian buku itu.

```

1. class Queue:
2.     def __init__(self):
3.         self._head = None # ujung depan (dequeue)
4.         self._tail = None # ujung belakang (enqueue)
5.         self._size = 0
6.
7.     def enqueue(self, data):      # O(1) — tempel ke tail
8.         baru = LLNode(data)
9.         if self._tail is None:
10.            self._head = self._tail = baru
11.        else:
12.            self._tail.next = baru
13.            self._tail = baru
14.            self._size += 1
15.
16.    def dequeue(self):            # O(1) — cabut head
17.        if self._head is None: return None
18.        data = self._head.data
19.        self._head = self._head.next
20.        if self._head is None: self._tail = None
21.        self._size -= 1
22.        return data
23.

```

3.2.1.1. CONTOH INPUT/INPUT CLI

```

>> PESAN NIM-001 ISBN-0003
[PESAN] Buku ISBN-0003 sedang TERSEDIA. Langsung gunakan PINJAM.
>> PESAN NIM-002 ISBN-0003
[PESAN] Buku ISBN-0003 sedang TERSEDIA. Langsung gunakan PINJAM.
>> PESAN NIM-003 ISBN-0003
[PESAN] Buku ISBN-0003 sedang TERSEDIA. Langsung gunakan PINJAM.

```

```

>> ANTRIAN ISBN-0003
[ANTRIAN] ISBN-0003 — 0 anggota mengantri:
(kosong)
Big-O: O(k) traversal antrian

```

3.2.2. Stack — Riwayat Transaksi & UNDO (modul_2.py)

Stack diimplementasikan dengan top = head Linked List sehingga push dan pop selalu bekerja di ujung depan tanpa traversal. Setiap transaksi disimpan sebagai dictionary berisi tx_id, aksi, nim, isbn, waktu, dan durasi. Saat BATALKAN_TERAKHIR dipanggil, CLI mem-pop transaksi teratas lalu membalik efeknya berdasarkan jenis aksi: PINJAM dikembalikan ke TERSEDIA, KEMBALIKAN dikembalikan ke DIPINJAM, PESAN dihapus dari antrian.

Untuk mencegah Stack tumbuh tak terbatas, ManajerRiwayat Dilengkapi metode arsip_transaksi_lama(), setelah Stack melebihi batas N elemen, elemen-elemen lama (terbawah) dipindahkan ke Linked List historis. Stack hanya menyisakan N elemen terbaru. Big-O archiving: $O(n)$.

```

1. class Stack:
2.     def push(self, data):      # O(1)
3.         baru = LLNode(data)
4.         baru.next = self._top
5.         self._top = baru
6.         self._size += 1
7.
8.     def pop(self):             # O(1)
9.         if self._top is None: return None

```

```

10. data = self._top.data
11. self._top = self._top.next
12. self._size -= 1
13. return data
14.

```

3.2.2.1. CONTOH INPUT/OUTPUT CLI

```

>> PINJAM NIM-001 ISBN-0005
[PINJAM] Berhasil. NIM-001 meminjam ISBN-0005 selama 14 hari. TX-0001
Big-O: O(log n) BST search + update
>> PINJAM NIM-002 ISBN-0007
[PINJAM] Berhasil. NIM-002 meminjam ISBN-0007 selama 14 hari. TX-0002
Big-O: O(log n) BST search + update

>> BATALKAN_TERAKHIR
[UNDO] Membatalkan: [TX-0002] PINJAM | NIM: NIM-002 | ISBN: ISBN-0007 | 2026-05-24 06:22:59
[UNDO] Status ISBN-0007 dikembalikan ke TERSEDIA.
Big-O: O(1) stack pop + O(log n) BST update

```

3.2.3. BST — Katalog Buku (modul_3.py)

BST menggunakan kunci ISBN string dengan perbandingan leksikografis. Fungsi `_insert_rekursif` mengembalikan boolean True/False untuk membedakan node baru (counter `_jumlah` naik) dari update duplikat (counter tetap). Tanpa mekanisme ini, insert ISBN yang sama dua kali akan menaikkan counter secara tidak benar.

Operasi delete menangani tiga kasus: (1) node daun — langsung hapus, (2) satu anak — ganti node dengan anaknya, (3) dua anak — salin data inorder successor (node terkecil di subtree kanan) ke node yang dihapus, lalu hapus inorder successor. Inorder traversal menghasilkan daftar buku menurut ISBN secara otomatis karena BST selalu mempertahankan invariant $kiri < root < kanan$.

```

1. class BSTKatalog:
2.     def insert(self, buku):      # O(log n) avg | O(n) worst
3.         if self._root is None:
4.             self._root = BSTNode(buku)
5.         else:
6.             ditambah = self._insert_rekursif(self._root, buku)
7.             if ditambah: self._jumlah += 1
8.
9.     def search(self, isbn):      # O(log n) avg | O(n) worst
10.        return self._search_rekursif(self._root, isbn)
11.
12.     def delete(self, isbn):      # O(log n) avg — 3 kasus
13.        # Kasus 1: daun | Kasus 2: satu anak | Kasus 3: dua anak
14.        # Kasus 3 menggunakan inorder successor (min di subtree kanan)
15.
16.     def inorder(self):          # O(n) — traversal seluruh pohon
17.        hasil = []
18.        self._inorder_rekursif(self._root, hasil)
19.        return hasil # menurut menaik berdasarkan ISBN
20.

```

3.2.3.1. CONTOH INPUT/OUTPUT CLI

```
>> KATALOG
[KATALOG] Total 80 buku (urutan ISBN):
No ISBN      Judul      Pengarang      Kategori      Status
-----
1 ISBN-0001  Digital Vol.1  Penulis-10     Sains         TERSEDIA
2 ISBN-0002  Data Vol.2    Penulis-5      Sains         TERSEDIA
3 ISBN-0003  Python Vol.3  Penulis-5      Fiksi         TERSEDIA
4 ISBN-0004  Elektronika Vol.4  Penulis-7     Teknik         TERSEDIA
5 ISBN-0005  Algoritma Vol.5  Penulis-14     Sains         TERSEDIA
6 ISBN-0006  Fisika Vol.6   Penulis-1      Teknik         TERSEDIA
7 ISBN-0007  Python Vol.7   Penulis-3      Teknik         TERSEDIA
8 ISBN-0008  Sinyal Vol.8   Penulis-14     Sains         TERSEDIA
9 ISBN-0009  Digital Vol.9  Penulis-12     Sains         TERSEDIA
10 ISBN-0010 Sinyal Vol.10  Penulis-18     Seni          TERSEDIA
11 ISBN-0011  Kontrol Vol.11 Penulis-12     Sejarah       TERSEDIA
12 ISBN-0012  Sistem Vol.12  Penulis-4      Teknik         TERSEDIA
13 ISBN-0013  Fisika Vol.13  Penulis-9      Sejarah       TERSEDIA
14 ISBN-0014  Elektronika Vol.14  Penulis-20     Sains         TERSEDIA
15 ISBN-0015  Sinyal Vol.15  Penulis-15     Seni          TERSEDIA
16 ISBN-0016  Python Vol.16  Penulis-10     Sains         TERSEDIA
17 ISBN-0017  Python Vol.17  Penulis-17     Teknik         TERSEDIA
18 ISBN-0018  Digital Vol.18  Penulis-12     Sejarah       TERSEDIA
19 ISBN-0019  Digital Vol.19  Penulis-20     Teknik         TERSEDIA
20 ISBN-0020  Kontrol Vol.20  Penulis-5      Seni          TERSEDIA
21 ISBN-0021  Sinyal Vol.21  Penulis-18     Sains         TERSEDIA
22 ISBN-0022  Fisika Vol.22  Penulis-8      Sains         TERSEDIA
23 ISBN-0023  Sistem Vol.23  Penulis-5      Fiksi         TERSEDIA
24 ISBN-0024  Kontrol Vol.24  Penulis-15     Sejarah       TERSEDIA
25 ISBN-0025  Algoritma Vol.25  Penulis-14     Fiksi         TERSEDIA
26 ISBN-0026  Data Vol.26    Penulis-5      Sejarah       TERSEDIA
27 ISBN-0027  Sinyal Vol.27  Penulis-9      Sains         TERSEDIA
28 ISBN-0028  Kontrol Vol.28  Penulis-9      Teknik         TERSEDIA
29 ISBN-0029  Data Vol.29    Penulis-12     Seni          TERSEDIA
30 ISBN-0030  Python Vol.30  Penulis-20     Sains         TERSEDIA
31 ISBN-0031  Sinyal Vol.31  Penulis-7      Seni          TERSEDIA
32 ISBN-0032  Algoritma Vol.32  Penulis-19     Seni          TERSEDIA
33 ISBN-0033  Algoritma Vol.33  Penulis-12     Seni          TERSEDIA
34 ISBN-0034  Fisika Vol.34  Penulis-11     Sains         TERSEDIA
35 ISBN-0035  Digital Vol.35  Penulis-7      Sains         TERSEDIA
36 ISBN-0036  Elektronika Vol.36  Penulis-4      Sains         TERSEDIA
37 ISBN-0037  Python Vol.37  Penulis-3      Sejarah       TERSEDIA
38 ISBN-0038  Fisika Vol.38  Penulis-18     Seni          TERSEDIA
39 ISBN-0039  Algoritma Vol.39  Penulis-15     Teknik         TERSEDIA
40 ISBN-0040  Jaringan Vol.40  Penulis-17     Fiksi         TERSEDIA
```

```
41 ISBN-0041  Python Vol.41  Penulis-10     Fiksi         TERSEDIA
42 ISBN-0042  Kontrol Vol.42  Penulis-10     Teknik         TERSEDIA
43 ISBN-0043  Fisika Vol.43  Penulis-20     Fiksi         TERSEDIA
44 ISBN-0044  Sinyal Vol.44  Penulis-17     Teknik         TERSEDIA
45 ISBN-0045  Algoritma Vol.45  Penulis-1      Teknik         TERSEDIA
46 ISBN-0046  Sistem Vol.46  Penulis-5      Sejarah       TERSEDIA
47 ISBN-0047  Elektronika Vol.47  Penulis-18     Seni          TERSEDIA
48 ISBN-0048  Sinyal Vol.48  Penulis-4      Sains         TERSEDIA
49 ISBN-0049  Digital Vol.49  Penulis-2      Sejarah       TERSEDIA
50 ISBN-0050  Elektronika Vol.50  Penulis-6      Sains         TERSEDIA
51 ISBN-0051  Digital Vol.51  Penulis-14     Sains         TERSEDIA
52 ISBN-0052  Digital Vol.52  Penulis-14     Seni          TERSEDIA
53 ISBN-0053  Data Vol.53    Penulis-12     Sains         TERSEDIA
54 ISBN-0054  Fisika Vol.54  Penulis-18     Sejarah       TERSEDIA
55 ISBN-0055  Data Vol.55    Penulis-12     Sejarah       TERSEDIA
56 ISBN-0056  Algoritma Vol.56  Penulis-11     Sejarah       TERSEDIA
57 ISBN-0057  Sinyal Vol.57  Penulis-3      Fiksi         TERSEDIA
58 ISBN-0058  Algoritma Vol.58  Penulis-20     Fiksi         TERSEDIA
59 ISBN-0059  Digital Vol.59  Penulis-7      Sejarah       TERSEDIA
60 ISBN-0060  Sinyal Vol.60  Penulis-18     Sejarah       TERSEDIA
61 ISBN-0061  Sistem Vol.61  Penulis-2      Teknik         TERSEDIA
62 ISBN-0062  Sinyal Vol.62  Penulis-2      Teknik         TERSEDIA
63 ISBN-0063  Python Vol.63  Penulis-2      Teknik         TERSEDIA
64 ISBN-0064  Python Vol.64  Penulis-2      Teknik         TERSEDIA
65 ISBN-0065  Python Vol.65  Penulis-2      Teknik         TERSEDIA
66 ISBN-0066  Python Vol.66  Penulis-2      Teknik         TERSEDIA
67 ISBN-0067  Python Vol.67  Penulis-2      Teknik         TERSEDIA
68 ISBN-0068  Python Vol.68  Penulis-2      Teknik         TERSEDIA
69 ISBN-0069  Python Vol.69  Penulis-2      Teknik         TERSEDIA
70 ISBN-0070  Python Vol.70  Penulis-2      Teknik         TERSEDIA
71 ISBN-0071  Python Vol.71  Penulis-2      Teknik         TERSEDIA
72 ISBN-0072  Python Vol.72  Penulis-2      Teknik         TERSEDIA
73 ISBN-0073  Python Vol.73  Penulis-2      Teknik         TERSEDIA
74 ISBN-0074  Python Vol.74  Penulis-2      Teknik         TERSEDIA
75 ISBN-0075  Python Vol.75  Penulis-2      Teknik         TERSEDIA
76 ISBN-0076  Python Vol.76  Penulis-2      Teknik         TERSEDIA
77 ISBN-0077  Python Vol.77  Penulis-2      Teknik         TERSEDIA
78 ISBN-0078  Sinyal Vol.78  Penulis-7      Seni          TERSEDIA
79 ISBN-0079  Kontrol Vol.79  Penulis-20     Fiksi         TERSEDIA
80 ISBN-0080  Elektronika Vol.80  Penulis-2      Fiksi         TERSEDIA

Big-O: O(n) inorder BST traversal
```

```
>> CARI_BUKU ISBN-0015
ISBN      : ISBN-0015
Judul     : Sinyal Vol.15
Pengarang : Penulis-15
Kategori  : Seni
Status    : TERSEDIA
Big-O: O(log n) BST search
```

3.2.4. Graph — Rekomendasi Ko-pinjam (modul_4.py)

Graf tak-berarah berbobot menggunakan adjacency list {isbn: [(isbn_tetangga, frekuensi)]}. Setiap kali dua buku dipinjam dalam satu sesi anggota yang sama, add_copinjam() dipanggil untuk menaikkan bobot edge. BFS menggunakan Queue Linked List buatan sendiri (bukan deque bawaan Python) dengan dua parameter kontrol: max_hop=2 (batasan kedalaman) dan min_bobot=1 (filter relevansi). Hasil rekomendasi diurutkan berdasarkan bobot kumulatif menggunakan insertion sort manual.

```
1. class GraphRekBuku:
2.     def add_copinjam(self, isbn_a, isbn_b): # O(deg)
3.         # update bobot kedua sisi
4.
5.     def rekomendasikan(self, isbn_sumber, max_hop=2): # O(V+E)
6.         antrian = Queue() # pakai Queue Linked List sendiri
7.         antrian.enqueue((isbn_sumber, 0, 0))
8.         dikunjungi = {isbn_sumber}
9.         # BFS terbatas max_hop
10.        # hasil diurutkan dengan insertion_sort_turun (O(k^2))
11.        # k = kandidat rekomendasi << n total buku
12.
```

3.2.4.1. CONTOH INPUT/OUTPUT CLI

```
>> PINJAM NIM-001 ISBN-0001
[PINJAM] Berhasil. NIM-001 meminjam ISBN-0001 selama 14 hari. TX-0003
Big-O: O(log n) BST search + update
>> PINJAM NIM-001 ISBN-0002
[PINJAM] Berhasil. NIM-001 meminjam ISBN-0002 selama 14 hari. TX-0004
Big-O: O(log n) BST search + update
>> PINJAM NIM-001 ISBN-0003
[PINJAM] Berhasil. NIM-001 meminjam ISBN-0003 selama 14 hari. TX-0005
Big-O: O(log n) BST search + update
```

```
>> REKOMENDASI ISBN-0001
[REKOMENDASI] Buku yang sering dipinjam bersama ISBN-0001:
1. ISBN-0005 | Algoritma Vol.5 | Bobot: 1 | Hop: 1 | DIPINJAM
2. ISBN-0002 | Data Vol.2 | Bobot: 1 | Hop: 1 | DIPINJAM
3. ISBN-0003 | Python Vol.3 | Bobot: 1 | Hop: 1 | DIPINJAM
Big-O: O(V+E) BFS graf ko-pinjam
```

3.2.5. Sorting — Laporan Bulanan (modul_5.py)

Shell Sort pada Linked List: Node-node dikumpulkan ke array pointer sementara $O(n)$, Shell Sort dengan gap sequence Knuth dijalankan pada array tersebut dengan swap data (bukan swap pointer), menghasilkan Linked List terurut tanpa mengubah struktur pointer. Big-O: $\sim O(n^{1.5})$.

Merge Sort pada Linked List: Split menggunakan teknik slow/fast pointer untuk menemukan tengah list $O(n)$, rekursif sort dua bagian, lalu merge dengan sentinel node dummy untuk menghindari penanganan head khusus. Murni pointer manipulation tanpa konversi ke array Python. Big-O: $O(n \log n)$.

```
1. def shell_sort_durasi(head_node): #  $\sim O(n^{1.5})$  | Ruang  $O(n)$ 
2.     nodes = [] # kumpulkan pointer node ke array
3.     gap = 1
4.     while gap < n // 3: gap = gap * 3 + 1 # Knuth sequence
5.     # Shell Sort dengan swap data (bukan pointer)
6.
7. def merge_sort_frekuensi(head_node): #  $O(n \log n)$  | Ruang  $O(\log n)$ 
8.     kiri, kanan = _split_tengah(head_node) # slow/fast pointer
9.     # rekursif sort, lalu merge dengan sentinel dummy node
10.
```

3.2.5.1. CONTOH INPUT/OUTPUT CLI

```
>> LAPORAN_BULAN

[LAPORAN] Peminjaman Berdasarkan Durasi (Shell Sort, descending):
No TX-ID    NIM      ISBN      Durasi (hari)
-----
1 TX-0005   NIM-001   ISBN-0003   14
2 TX-0004   NIM-001   ISBN-0002   14
3 TX-0003   NIM-001   ISBN-0001   14
4 TX-0001   NIM-001   ISBN-0005   14
Big-O Shell Sort:  $\sim O(n^{1.5})$ 

[LAPORAN] Frekuensi Peminjaman per Judul (Merge Sort, descending):
No ISBN      Frekuensi
-----
1 ISBN-0003   1
2 ISBN-0002   1
3 ISBN-0001   1
4 ISBN-0005   1
Big-O Merge Sort:  $O(n \log n)$ 

[LAPORAN] Tabel Runtime Sorting
N | Shell Sort (s) | Merge Sort (s)
-----
4 | 0.000013 | 0.000015
4 | 0.000009 | 0.000011
4 | 0.000008 | 0.000009
Big-O Shell:  $\sim O(n^{1.5})$  | Big-O Merge:  $O(n \log n)$ 
```

3.2.6. CLI — Loop Utama (modul_6.py)

CLI menggunakan pola dispatch table: dictionary yang memetakan string perintah ke method handler. Pendekatan ini menghindari if-elif panjang, memudahkan penambahan perintah baru, dan setiap handler dapat diuji secara terpisah. Setiap handler memvalidasi argumen, memanggil modul yang tepat, dan mencetak Big-O operasi setelah eksekusi.

Perintah CLI	Big-O Operasi	Modul yang Dipanggil
CARI_BUKU <isbn>	$O(\log n)$ BST search	modul_3
PINJAM <nim> <isbn>	$O(\log n)$ BST + $O(1)$ Stack	modul_3, modul_2, modul_4
KEMBALIKAN <isbn>	$O(\log n)$ BST + $O(1)$ dequeue	modul_3, modul_1, modul_2
PESAN <nim> <isbn>	$O(1)$ enqueue + $O(k)$ cek duplikat	modul_1, modul_2
BATALKAN_TERAKHIR	$O(1)$ pop + $O(\log n)$ BST revert	modul_2, modul_3
REKOMENDASI <isbn>	$O(V+E)$ BFS	modul_4
ANTRIAN <isbn>	$O(k)$ traversal antrian	modul_1
KATALOG	$O(n)$ inorder BST	modul_3
LAPORAN_BULAN	$\sim O(n^{1.5})$ Shell / $O(n \log n)$ Merge	modul_5

3.2.6.1. CONTOH INPUT/OUTPUT CLI

DAFTAR PERINTAH	Big-O
CARI_BUKU <isbn>	$O(\log n)$
PINJAM <nim> <isbn>	$O(\log n)$
KEMBALIKAN <isbn>	$O(\log n)$
PESAN <nim> <isbn>	$O(1)$ enqueue
BATALKAN_PESAN <nim> <isbn>	$O(k)$ rebuild
BATALKAN_TERAKHIR	$O(1)$ pop
REKOMENDASI <isbn>	$O(V+E)$ BFS
ANTRIAN <isbn>	$O(k)$ traversal
KATALOG	$O(n)$ inorder
LAPORAN_BULAN	$O(n^{1.5})/O(n \log n)$
BANTUAN	$O(1)$
KELUAR	-

BAB IV

ANALISIS BIG-O

4.1. ANALISIS KOMPLEKSITAS RUANG DAN WAKTU SETIAP FUNGSI UTAMA

Struktur Data	Operasi	Big-O Waktu	Big-O Ruang	Kasus
Queue (Linked List)	enqueue	$O(1)$	$O(1)$ per op	Semua kasus
Queue (Linked List)	dequeue	$O(1)$	$O(1)$ per op	Semua kasus
Queue (Linked List)	tampilkan_antrian	$O(k)$	$O(k)$	k = panjang antrian
Queue (Linked List)	batalan_pesanan (rebuild)	$O(k)$	$O(k)$	k = panjang antrian
Stack (Linked List)	push	$O(1)$	$O(1)$ per op	Semua kasus
Stack (Linked List)	pop / peek	$O(1)$	$O(1)$ per op	Semua kasus
Stack (Linked List)	tampilkan_stack	$O(n)$	$O(n)$	n = total transaksi
Stack (Linked List)	archiving	$O(n)$	$O(n)$	n = total transaksi
Linked List (dasar)	tambah_depan	$O(1)$	$O(1)$ per op	Semua kasus
Linked List (dasar)	tambah_belakang	$O(1)$	$O(1)$ per op	Dengan pointer tail
Linked List (dasar)	cari / hapus_nilai	$O(n)$	$O(1)$	Traversal linear
BST	insert / search / delete	$O(\log n)$	$O(\log n)$ call stack	Rata-rata (data acak)

Struktur Data	Operasi	Big-O Waktu	Big-O Ruang	Kasus
BST	insert / search / delete	$O(n)$	$O(n)$ call stack	Worst-case (data terurut)
BST	update_status	$O(\log n)$	$O(\log n)$	Rata-rata
BST	inorder traversal	$O(n)$	$O(n)$	Semua kasus
BST	hitung_tinggi	$O(n)$	$O(n)$	Traversals seluruh pohon
Graph	tambah_vertex	$O(1)$	$O(1)$	Semua kasus
Graph	add_copinjam	$O(\text{deg})$	$O(1)$ per op	deg = derajat node
Graph BFS	rekomendasikan	$O(V+E)$	$O(V)$	V = vertex, E = edge
Shell Sort (LL)	sorting durasi	$\sim O(n^{1.5})$	$O(n)$ ptr sementara	Gap Knuth
Merge Sort (LL)	sorting frekuensi	$O(n \log n)$	$O(\log n)$ rekursif	Semua kasus

4.1.1. Queue — Antrian Pemesanan Buku

Queue diimplementasikan berbasis Singly Linked List dengan dua pointer: `_head` (ujung depan) dan `_tail` (ujung belakang). Pointer `_tail` dipertahankan agar operasi enqueue tidak memerlukan traversal ke akhir list, sehingga tetap $O(1)$. Tanpa pointer tail, setiap enqueue akan menjadi $O(n)$. **Enqueue $O(1)$:** Node baru langsung disambung ke `_tail`, lalu `_tail` digeser. Tidak ada traversal. **Dequeue $O(1)$:** Head langsung dicabut dan digeser ke node berikutnya. Jika antrian kosong setelah dequeue, `_tail` juga direset ke None untuk menghindari dangling pointer.

batalan_pesanan $O(k)$: Queue tidak mendukung random delete, sehingga perlu rebuild yaitu:

1. semua elemen diambil ke list sementara $O(k)$.
2. elemen target difilter.
3. lalu Queue baru dibangun.

Dengan k = panjang antrian buku tersebut.

4.1.1.1. CODE FUNGSI UTAMA

```
1. def enqueue(self, data):
2.     baru = LLNode(data)
3.
4.     if self._tail is None:
5.         self._head = baru
6.         self._tail = baru
7.     else:
8.         self._tail.next = baru
9.         self._tail = baru
10.
11.     self._size += 1
12.
13.
14. def dequeue(self):
15.     if self._head is None:
16.         return None
17.
18.     data_keluar = self._head.data
19.     self._head = self._head.next
20.
21.     if self._head is None:
22.         self._tail = None
23.
24.     self._size -= 1
25.     return data_keluar
26.
27.
28. def _handle_batalkan_pesan(self, token):
29.
30.     if len(token) < 3:
31.         print('[ERROR] Penggunaan: BATALKAN_PESAN <nim> <isbn>')
32.         return
33.
34.     nim = token[1].upper()
35.     isbn = token[2].upper()
36.
37.     hasil = self._antrian.batalikan_pesan(isbn, nim)
38.
39.     print(hasil['pesan'])
40.     print(f' Big-O: {hasil["big_o"]}')
41.
42.     if hasil['berhasil']:
43.         self._riwayat.catat('BATAL_PESAN', nim, isbn)
44.
```

batalikan_pesan bukan method bawaan pada queue.py tetapi merupakan bagian dari proses traversal seluruh queue, membangun ulang queue tanpa anggota yang dibatalkan.

4.1.1.2. TABEL ANALISIS KOMPLEKSITAS RUANG DAN WAKTU

Struktur Data	Operasi	Big-O Waktu	Big-O Ruang	Kasus
Queue (Linked List)	enqueue	$O(1)$	$O(1)$ per op	Semua kasus
Queue (Linked List)	dequeue	$O(1)$	$O(1)$ per op	Semua kasus
Queue (Linked List)	tampilkan_antrian	$O(k)$	$O(k)$	k = panjang antrian
Queue (Linked List)	batalikan_pesanan (rebuild)	$O(k)$	$O(k)$	k = panjang antrian

4.1.2. Stack — Riwayat Transaksi & UNDO

Stack diimplementasikan dengan $\text{top} = \text{head}$ Linked List. Setiap operasi push/pop hanya menyentuh node paling depan tanpa traversal, sehingga $O(1)$ terjamin. Strategi archiving disediakan untuk mencegah Stack tumbuh tak terbatas.

Push $O(1)$: Node baru disisipkan di depan (top). Pointer top digeser.

Pop $O(1)$: Top dicabut dan pointer top digeser ke node di bawahnya.

Archiving $O(n)$: Saat Stack melebihi batas `maks_simpan`, elemen lama dipindah ke Linked List historis (`modul_7`). Total traversal satu kali $O(n)$, amortized cost per transaksi $O(1)$.

4.1.2.1. CODE FUNGSI UTAMA

```
1. def push(self, data):
2.     baru = LLNode(data)
3.     baru.next = self._top
4.     self._top = baru
5.     self._size += 1
6.
7.
8. def pop(self):
9.     if self._top is None:
10.         return None
11.
12.     data_keluar = self._top.data
13.     self._top = self._top.next
14.     self._size -= 1
15.     return data_keluar
16.
17.
18. def arsip_transaksi_lama(self, maks_simpan=50):
19.
20.     if self._size <= maks_simpan:
21.         return []
22.
23.     semua = self.tampilkan_stack()
24.
25.     tetap = semua[:maks_simpan]
26.     arsip = semua[maks_simpan:]
27.
```

```

28. self._top = None
29. self._size = 0
30.
31. for item in reversed(tetap):
32.     self.push(item)
33.
34. return arsip

```

4.1.2.2. TABEL ANALISIS KOMPLEKSITAS RUANG DAN WAKTU

Stack (Linked List)	push	O(1)	O(1) per op	Semua kasus
Stack (Linked List)	pop / peek	O(1)	O(1) per op	Semua kasus
Stack (Linked List)	tampilkan_stack	O(n)	O(n)	n = total transaksi
Stack (Linked List)	archiving	O(n)	O(n)	n = total transaksi

4.1.3. BST — Katalog Buku

BST menggunakan ISBN string sebagai kunci dengan perbandingan leksikografis. Kinerja BST sangat bergantung pada urutan data yang dimasukkan.

Rata-rata $O(\log n)$: Jika 80 buku disisipkan secara acak (seed=13), tinggi pohon $\approx \log_2(80) \approx 7$ level. Setiap operasi hanya menelusuri satu cabang dari akar ke daun.

Worst-case $O(n)$: Jika data disisipkan secara terurut leksikografis (ISBN-0001, ISBN-0002, ...), setiap node baru selalu masuk ke cabang kanan sehingga pohon menjadi linear (miring). Tinggi = n, semua operasi menjadi $O(n)$. Dibuktikan empiris: tinggi = 300 untuk N=300 data terurut.

Insert duplikat: Fungsi `_insert_rekursif` mengembalikan bool True/False untuk membedakan node baru (counter_jumlah naik) vs update duplikat (counter tetap). Tanpa mekanisme ini, insert ISBN yang sama dua kali akan menaikkan counter secara keliru.

Delete tiga kasus: (1) Node daun: langsung hapus. (2) Satu anak: ganti dengan anaknya. (3) Dua anak: cari inorder successor (node paling kiri di subtree kanan), salin datanya, lalu hapus successor.

4.1.3.1. CODE FUNGSI UTAMA

```

1. def insert(self, buku):
2.     if self._root is None:
3.         self._root = BSTNode(buku)
4.         self._jumlah += 1
5.     else:
6.         ditambah = self._insert_rekursif(self._root, buku)
7.         if ditambah:
8.             self._jumlah += 1
9.
10.
11. def delete(self, isbn):

```

```

12. root_baru, berhasil = self._delete_rekursif(
13.     self._root,
14.     isbn
15. )
16.
17. if berhasil:
18.     self._root = root_baru
19.     self._jumlah -= 1
20.
21. return berhasil
22.

```

4.1.3.2. TABEL ANALISIS KOMPLEKSITAS RUANG DAN WAKTU

BST	insert / search / delete	$O(\log n)$	$O(\log n)$ call stack	Rata-rata (data acak)
BST	insert / search / delete	$O(n)$	$O(n)$ call stack	Worst-case (data terurut)
BST	update_status	$O(\log n)$	$O(\log n)$	Rata-rata
BST	inorder traversal	$O(n)$	$O(n)$	Semua kasus
BST	hitung_tinggi	$O(n)$	$O(n)$	Traversals seluruh pohon

4.1.4. Graph — Rekomendasi Ko-pinjam

Graf tak-berarah berbobot menggunakan adjacency list {isbn: [(isbn_tetangga, frekuensi)]}. Representasi ini dipilih karena sistem rekomendasi buku bersifat sparse (kebanyakan buku tidak terhubung langsung), sehingga $O(V+E)$ jauh lebih efisien dari $O(V^2)$ adjacency matrix.

add_copinjam $O(\deg)$: Perlu scan adjacency list node untuk cek apakah edge sudah ada. Jika ya, naikan bobot +1. Jika belum, tambahkan entry baru. Dijalankan untuk kedua sisi (tidak berarah).

BFS $O(V+E)$: BFS menjelajah level per level hingga $\text{max_hop}=2$. Visited set mencegah kunjungan ganda. Queue Linked List sendiri (bukan deque bawaan) digunakan sebagai antrian BFS. Filter min_bobot mengurangi E efektif yang dijelajahi.

4.1.4.1. CODE FUNGSI UTAMA

```

1. def add_copinjam(self, isbn_a, isbn_b):
2.
3.     if isbn_a == isbn_b:
4.         return
5.
6.     self.tambah_vertex(isbn_a)
7.     self.tambah_vertex(isbn_b)
8.
9.     self._adj[isbn_a] = self._update_bobot(
10.         self._adj[isbn_a],
11.         isbn_b
12.     )

```

```

13.
14.     self._adj[isbn_b] = self._update_bobot(
15.         self._adj[isbn_b],
16.         isbn_a
17.     )
18.
19.
20. def rekomendasikan(
21.     self,
22.     isbn_sumber,
23.     max_hop=2,
24.     min_bobot=1
25. ):
26.
27.     if isbn_sumber not in self._adj:
28.         return []
29.
30.     antrian = Queue()
31.     antrian.enqueue((isbn_sumber, 0, 0))
32.
33.     dikunjungi = {isbn_sumber}
34.     hasil = []
35.
36.     while not antrian.is_empty():
37.
38.         isbn_kini, hop_kini, bobot_kumulatif = (
39.             antrian.dequeue()
40.         )
41.
42.         for isbn_tetangga, bobot_edge in (
43.             self._adj.get(isbn_kini, [])
44.         ):
45.
46.             if isbn_tetangga in dikunjungi:
47.                 continue
48.
49.             if bobot_edge < min_bobot:
50.                 continue
51.
52.             dikunjungi.add(isbn_tetangga)
53.
54.             hop_baru = hop_kini + 1
55.             bobot_baru = (
56.                 bobot_kumulatif + bobot_edge
57.             )
58.
59.             hasil.append(
60.                 (isbn_tetangga, bobot_baru, hop_baru)
61.             )
62.
63.             if hop_baru < max_hop:
64.                 antrian.enqueue(
65.                     (
66.                         isbn_tetangga,
67.                         hop_baru,
68.                         bobot_baru
69.                     )
70.                 )
71.
72.     hasil = self._insertion_sort_turun(hasil)
73.
74.     return hasil
75.

```

4.1.4.2. TABEL ANALISIS KOMPLEKSITAS RUANG DAN WAKTU

Graph	tambah_vertex	$O(1)$	$O(1)$	Semua kasus
Graph	add_copinjam	$O(\text{deg})$	$O(1)$ per op	deg = derajat node
Graph BFS	rekomendasikan	$O(V+E)$	$O(V)$	V = vertex, E = edge

4.1.5. Shell Sort & Merge Sort pada Linked List

Shell Sort $\sim O(n^{1.5})$: Menggunakan gap sequence Knuth (1, 4, 13, 40, ...). Node-node dikumpulkan ke array pointer sementara $O(n)$, Shell Sort dijalankan dengan swap data (bukan pointer). Efektif untuk data hampir terurut karena elemen yang jauh dapat dipindahkan lebih cepat daripada Insertion Sort biasa.

Merge Sort $O(n \log n)$: Murni pointer manipulation: split menggunakan slow/fast pointer $O(n)$ untuk temukan tengah, rekursif sort dua bagian, merge dengan sentinel node dummy untuk hindari penanganan head khusus. Guaranteed $O(n \log n)$ untuk semua kasus.

4.1.5.1. CODE FUNGSI UTAMA

```
1. def shell_sort_durasi(head_node):
2.
3.     nodes = []
4.     cur = head_node
5.
6.     while cur is not None:
7.         nodes.append(cur)
8.         cur = cur.next
9.
10.    n = len(nodes)
11.
12.    if n <= 1:
13.        return head_node
14.
15.    gap = 1
16.    while gap < n // 3:
17.        gap = gap * 3 + 1
18.
19.    while gap >= 1:
20.        for i in range(gap, n):
21.
22.            temp_data = nodes[i].data
23.            j = i
24.
25.            while (
26.                j >= gap and
27.                nodes[j - gap].data.get('durasi', 0)
28.                < temp_data.get('durasi', 0)
29.            ):
30.                nodes[j].data = nodes[j - gap].data
31.                j -= gap
```



```

32.
33.     nodes[j].data = temp_data
34.
35.     gap //= 3
36.
37.     return head_node
38.
39.
40. def merge_sort_frekuensi(head_node):
41.
42.     if head_node is None or head_node.next is None:
43.         return head_node
44.
45.     kiri, kanan = _split_tengah(head_node)
46.
47.     kiri = merge_sort_frekuensi(kiri)
48.     kanan = merge_sort_frekuensi(kanan)
49.
50.     return _merge_descending(kiri, kanan)
51.

```

4.1.5.2. TABEL ANALISIS KOMPLEKSITAS RUANG DAN WAKTU

Shell Sort (LL)	sorting durasi	$\sim O(n^{1.5})$	$O(n)$ ptr sementara	Gap Knuth
Merge Sort (LL)	sorting frekuensi	$O(n \log n)$	$O(\log n)$ rekursif	Semua kasus

4.2. TABEL RINGKASAN

Struktur Data / Operasi	Big-O Teori	Hasil Empiris N=20	Hasil Empiris N=80	Hasil Empiris N=300	Konfirmasi
Queue enqueue N	$O(n)$ total	0.000013 s	0.000032 s	0.000123 s	Sesuai — linear
Queue dequeue N	$O(n)$ total	0.000015 s	0.000065 s	0.000202 s	Sesuai — linear
Stack push N	$O(n)$ total	0.000012 s	0.000029 s	0.000136 s	Sesuai — linear
Stack pop N	$O(n)$ total	0.000015 s	0.000055 s	0.000172 s	Sesuai — linear
BST insert acak	$O(n \log n)$	0.000093 s	0.000379 s	0.000905 s	Sesuai — sublinear

Struktur Data / Operasi	Big-O Teori	Hasil Empiris N=20	Hasil Empiris N=80	Hasil Empiris N=300	Konfirmasi
BST insert terurut	$O(n^2)$ worst	0.000132 s	0.001015 s	0.014129 s	Sesuai — pertumbuhan cepat
BST search	$O(\log n)$ avg	0.000002 s	0.000001 s	0.000003 s	Sesuai — hampir konstan
BST inorder	$O(n)$	0.000009 s	0.000022 s	0.000067 s	Sesuai — linear
Graph BFS	$O(V+E)$	0.000021 s	0.000034 s	0.000058 s	Sesuai — lambat naik
Shell Sort (LL)	$\sim O(n^{1.5})$	0.000046 s	0.000301 s	0.002506 s	Sesuai — pertumbuhan sedang
Merge Sort (LL)	$O(n \log n)$	0.000039 s	0.000166 s	0.000357 s	Sesuai — lebih cepat dari Shell

4.3. PERBANDINGAN TEORITIS VS EMPIRIS

Berdasarkan tabel ringkasan di atas, operasi yang paling kritis secara performa adalah BST search/insert ($O(\log n)$ rata-rata vs $O(n)$ worst-case) dan BFS Graf ($O(V+E)$). Eksperimen runtime pada Bab V memverifikasi bahwa hasil empiris konsisten dengan prediksi teoritis: BST dengan data acak menunjukkan tinggi pohon $\approx \log_2(n)$, sementara data terurut menghasilkan tinggi = n (pohon linear).

BST insert terurut menunjukkan pertumbuhan yang sangat signifikan (0.000132 s $\rightarrow$ 0.014129 s, naik ~ 107 kali lipat dari $N=20$ ke $N=300$), mengkonfirmasi worst-case $O(n^2)$ secara empiris. Sebaliknya, BST insert acak hanya naik ~ 10 kali lipat, konsisten dengan $O(n \log n)$.

BAB V

HASIL EKSPERIMEN

5.1. SETUP EKSPERIMEN

Eksperimen runtime dijalankan menggunakan script `experiments/benchmark.py` pada komputer pengembang dengan konfigurasi berikut:

- Bahasa: Python 3.x
- Random seed: 13 (tetap, agar hasil reproduisibel)
- Pengukuran waktu: `time.perf_counter()` — presisi tinggi
- Pengulangan: rata-rata 5 ulangan per operasi untuk meminimalkan noise
- Ukuran dataset: $N = 20$ (kecil), $N = 80$ (sesuai parameter sistem), $N = 300$ (beban tinggi)
- Semua struktur data diimplementasikan dari nol tanpa library bawaan Python

5.2. TABEL RUNTIME 3+ UKURAN DATA

5.2.1. Hasil Benchmark Queue & Stack

Queue dan Stack keduanya berbasis Linked List dengan operasi $O(1)$ per elemen. Tabel berikut menunjukkan total waktu untuk N operasi berturut-turut.

Struktur Data	Operasi	N = 20	N = 80	N = 300
Queue	enqueue N elemen (s)	0.000014	0.000035	0.000134
Queue	dequeue N elemen (s)	0.000018	0.000060	0.000218
Stack	push N elemen (s)	0.000019	0.000034	0.000120
Stack	pop N elemen (s)	0.000016	0.000049	0.000206

Diskusi: Rasio waktu $N=20$ ke $N=300$ berkisar 9–14 kali, sementara rasio N -nya adalah 15 kali. Ini konsisten dengan $O(n)$ total untuk N operasi $O(1)$. Sedikit perbedaan dari faktor $15\times$ disebabkan oleh overhead Python interpreter dan cache CPU yang lebih efisien pada iterasi berulang.

5.2.2. Hasil Benchmark BST

BST adalah struktur data dengan kinerja paling bervariasi tergantung urutan data. Eksperimen membandingkan dua skenario: data acak (rata-rata) dan data terurut leksikografis (worst-case).

Operasi BST	N = 20	N = 80	N = 300	Keterangan
Insert data acak (s)	0.0000104	0.000298	0.0001084	$O(n \log n)$ — naik perlahan
Insert data terurut (s)	0.00079	0.001016	0.021517	$O(n^2)$ worst — naik drastis
Search satu elemen (s)	0.000001	0.000002	0.000005	$O(\log n)$ — hampir konstan
Inorder traversal (s)	0.000007	0.000022	0.000076	$O(n)$ — linear
Tinggi pohon (data acak)	10	12	16	$\approx \log_2(n)$: 4.3, 6.3, 8.2
Tinggi pohon (data terurut)	20	80	300	= n (pohon miring, worst-case)

Diskusi: Hasil BST insert terurut mengkonfirmasi secara empiris fenomena worst-case BST. Pada $N=300$, insert terurut $15.6\times$ lebih lambat dari insert acak (0.014129 s vs 0.000905 s). Tinggi pohon terurut = N (300) berbanding tinggi pohon acak = 16, membuktikan bahwa data acak menghasilkan pohon yang jauh lebih seimbang (mendekati $O(\log n)$). Untuk sistem perpustakaan nyata dengan seed=13, data buku digenerate secara acak sehingga BST beroperasi di kasus rata-rata $O(\log n)$.

5.2.3. Hasil Benchmark Graph BFS

Graf menggunakan representasi adjacency list. Setiap eksperimen mensimulasikan N buku dengan ko-pinjam acak sebanyak $N\times 2$ transaksi, lalu BFS dijalankan dari satu sumber.

Parameter	N = 20 buku	N = 80 buku	N = 300 buku
Jumlah vertex (V)	20	78	293
Jumlah edge (E)	33	155	598
Derajat rata-rata	3.30	3.97	4.08
add_copinjam N×2 (s)	0.000092	0.000329	0.001697
BFS rekomendasikan (s)	0.000024	0.000040	0.0000122

Diskusi: BFS menunjukkan pertumbuhan yang sangat lambat ($0.000021 \text{ s} \rightarrow 0.000058 \text{ s}$ untuk V naik dari 20 ke 293), konsisten dengan $O(V+E)$. Derajat rata-rata stabil di sekitar 4, yang berarti BFS dari satu sumber dengan $\text{max_hop}=2$ hanya menjelajahi sekitar $1+4+12=17$ node dari 80 total node sistem, menjadikan rekomendasi sangat efisien. Vertex yang ditemukan sedikit lebih kecil dari N karena tidak semua ISBN selalu terhubung (beberapa buku mungkin tidak dipinjam bersama).

5.2.4. Hasil Benchmark Sorting pada Linked List

Shell Sort dan Merge Sort keduanya diimplementasikan langsung pada Linked List tanpa konversi ke array Python. Sorting digunakan pada laporan bulanan untuk dua keperluan berbeda.

Algoritma	N = 20	N = 80	N = 300	Big-O Teori
Shell Sort (durasi pinjam)	0.0000844	0.0001271	0.004370	$\sim O(n^{1.5})$
Merge Sort (frekuensi ISBN)	0.000070	0.000299	0.000570	$O(n \log n)$

Diskusi: Perbandingan empiris menunjukkan bahwa Merge Sort konsisten lebih cepat dari Shell Sort pada $N=80$ dan $N=300$. Rasio pertumbuhan Shell Sort ($54.5\times$) lebih besar dari Merge Sort ($9.2\times$) untuk kenaikan N dari 20 ke 300, mengkonfirmasi bahwa $O(n^{1.5})$ tumbuh lebih cepat

dari $O(n \log n)$. Pada $N=20$ Shell Sort sedikit lebih cepat (0.000046 vs 0.000039) karena overhead rekursif Merge Sort lebih terasa pada data kecil. Untuk sistem perpustakaan dengan N transaksi bulanan ≤ 300 , keduanya masih sangat cepat (< 3 ms).

5.2.5. Hasil Benchmark Integrasi 300 Event

Simulasi end-to-end 300 event campuran (PINJAM, KEMBALIKAN, PESAN, REKOMENDASI) dijalankan untuk mengukur kinerja sistem secara keseluruhan pada beban minimum yang disyaratkan panduan.

Parameter	Nilai	Keterangan
Total event	300	Campuran PINJAM, KEMBALIKAN, PESAN, REKOMENDASI
Total waktu	0.0032 detik	Keseluruhan 300 event selesai
Transaksi PINJAM/KEMBALIKAN	88	29.3% dari 300 event berhasil diproses
Ukuran Stack akhir	88	Semua transaksi tercatat di Stack riwayat
Big-O dominan per event	$O(\log n)$	BST search/update mendominasi setiap event
Waktu rata-rata per event	~ 0.0000107 detik	$300 \text{ event} / 0.0032 \text{ s} = \sim 10.7 \mu\text{s}$ per event

Diskusi: Sistem berhasil memproses 300 event campuran dalam 3.2 milidetik, jauh di bawah batas waktu respons yang dapat diterima untuk sistem perpustakaan interaktif. Big-O dominan $O(\log n)$ dari operasi BST terbukti sangat efisien untuk koleksi 80 buku. Catatan: Graf menunjukkan vertex=0 karena pada simulasi integrasi ini ko-pinjam belum dicatat (anggota dalam satu event tidak selalu pinjam lebih dari satu buku secara bersamaan).

5.2.6. Perbandingan Shell Sort vs Merge Sort — Visualisasi Tren

Tabel berikut merangkum tren pertumbuhan waktu untuk memudahkan perbandingan visual antara dua algoritma sorting:

N	Shell Sort (s)	Merge Sort (s)	Selisih (Shell - Merge)	Merge lebih cepat?
20	0.000046	0.000039	0.000007	Ya (1.18×)
80	0.000301	0.000166	0.000135	Ya (1.81×)
300	0.002506	0.000357	0.002149	Ya (7.02×)

Kesimpulan perbandingan: Merge Sort semakin unggul seiring bertambahnya N. Pada N=20 selisihnya kecil (1.18×), namun pada N=300 Merge Sort sudah 7× lebih cepat. Ini mengkonfirmasi secara empiris bahwa $O(n \log n)$ tumbuh lebih lambat dari $\sim O(n^{1.5})$ untuk N besar. Untuk laporan bulanan perpustakaan ($N \approx$ jumlah transaksi per bulan), Merge Sort adalah pilihan yang lebih skalabel.

5.4. RINGKASAN DISKUSI KESESUAIAN

Dari seluruh eksperimen yang dijalankan, semua hasil empiris konsisten dengan prediksi Big-O teoritis:

- Queue dan Stack: pertumbuhan linear $O(n)$ total untuk N operasi $O(1)$ — terkonfirmasi
- BST acak: pertumbuhan $O(n \log n)$ untuk insert, $O(\log n)$ untuk search — terkonfirmasi
- BST teratur: pertumbuhan $O(n^2)$ untuk insert (worst-case) — terkonfirmasi dengan selisih dramatis
- Graph BFS: pertumbuhan lambat $O(V+E)$, efisien untuk sparse graph — terkonfirmasi
- Shell Sort: $\sim O(n^{1.5})$ — pertumbuhan lebih cepat dari Merge Sort, terkonfirmasi
- Merge Sort: $O(n \log n)$ guaranteed — pertumbuhan paling konsisten, terkonfirmasi
- Integrasi 300 event: sistem responsif dengan latency $\sim 10.7 \mu s$ per event — terkonfirmasi

BAB VI

JAWABAN PERTANYAAN ANALISIS

Bab ini menjawab kelima pertanyaan analisis wajib sesuai panduan Topik 6, didukung data eksperimen dari Bab V.

6.1. Pertanyaan 1

BST Katalog dengan kunci ISBN string: jika 80 buku disisipkan dalam urutan teratur leksikografis (ISBN-0001, ISBN-0002, ..., ISBN-0080), BST menjadi linear. Hitung tinggi tree

worst-case vs rata-rata (data acak). Untuk $N=10.000$ judul perpustakaan universitas, berapa operasi search?

Bandingkan dengan implementasi Hash Table $O(1)$ average dan jelaskan kapan BST tetap lebih unggul.

➤ Analisis Tinggi Pohon

Perlu di ketahui bahwa performa BST sangat di pengaruhi oleh bentuk pohon yang terbentuk saat proses insert data. Ketika 80 buku disisipkan dengan urutan leksikografis menaik (ISBN-0001, ISBN-0002, ..., ISBN-0080), karena setiap ISBN baru selalu lebih besar dari semua ISBN sebelumnya, maka setiap node baru selalu masuk ke cabang kanan. Akibatnya BST berubah menjadi Pohon linear (degenerate tree) yang menyerupai linked list dengan struktur miring ke kanan dengan tinggi $h = 80 = n$.

- **Tinggi worst-case (terurut): $h = n = 80$**

Dampaknya terhadap operasi BST yaitu search, insert, delete kompleksitasnya menjadi $O(n)$ karena traversal harus melewati node satu persatu. Disini BST kehilangan efisiensi logaritmiknya karena tidak lagi bercabang secara seimbang.

- **Tinggi rata-rata (acak, seed=13): $h = \log_2(80) = 6.32$**

Sehingga didapatkan tinggi rata rata BST sekitar 6-7 level. Dampaknya terhadap operasi BST yaitu search, insert, delete kompleksitasnya menjadi $O(\log n)$ karena setiap langkah mempersempit pencarian menjadi setengah bagian, ini jauh lebih efisien dari pada worst-case.

➤ Operasi Search untuk $N = 10.000$

Untuk $N = 10.000$ judul perpustakaan universitas:

- BST acak (rata-rata):

search membutuhkan $= \log_2(10.000) = 13.29$, sehingga di dapatkan bahwa operasi search hanya membutuhkan sekitar 13 perbandingan dengan kompleksitas $O(\log n)$

- BST terurut (worst-case):

search membutuhkan hingga 10.000 perbandingan karena proses insert terurut dan BST menjadi miring. Kompleksitasnya menjadi $O(n)$ karena harus traversal ke setiap node.

- Hash Table (average):

search membutuhkan $O(1)$ rata-rata, tidak bergantung N (tinggi pohon) karena data langsung diakses melalui fungsi Hash tanpa traversal pohon. Ini cocok untuk lookup langsung ISBN tertentu, pencarian exact key sangat cepat.

➤ Kapan BST Tetap Lebih Unggul dari Hash Table?

Meskipun Hash Table $O(1)$ lebih cepat untuk single point query, BST tetap lebih unggul dalam tiga skenario utama:

- Range query: misal 'tampilkan semua buku dengan ISBN antara ISBN-0020 dan ISBN-0050'. BST dapat melakukan inorder traversal parsial secara efisien dengan kompleksitas $O(k)$ dimana k adalah jumlah data hasil. Hash Table tidak mendukung operasi ini karena menyimpan data secara terurut dan akibatnya harus memeriksa hampir seluruh data.
- Data terurut otomatis: inorder traversal BST menghasilkan daftar buku terurut ISBN dalam urutan ascending (kecil ke besar) tanpa sorting tambahan $O(n)$, Hash Table membutuhkan sorting tambahan sehingga kompleksitasnya $O(n \log n)$.
- Operasi predecessor/successor: 'buku dengan ISBN tepat sebelum/sesudah ISBN-0030'. Kompleksitas BST $O(\log n)$ sedangkan Hash Table tidak mendukung secara natural (karena tidak menyimpan data secara terurut)
- Tidak ada collision: BST tidak menggunakan mekanisme hashing sehingga tidak mengalami collision seperti pada Hash Table. Sedangkan Pada Hash Table, collision terjadi ketika beberapa key masuk ke index yang sama dan dapat menyebabkan performa turun hingga $O(n)$ pada worst-case.

6.2. Pertanyaan 2

Queue antrian pemesanan per-buku: jika sebuah buku populer memiliki 50 anggota dalam antrian dan buku dikembalikan rata-rata 3 hari sekali, estimasikan waktu tunggu anggota ke-50 dalam hari. Bandingkan dengan pendekatan Priority Queue (anggota premium didahulukan): hitung biaya tambahan operasi enqueue $O(n)$ vs queue biasa $O(1)$ untuk $N=50$ anggota.

Estimasi Waktu Tunggu Anggota ke-50

Parameter: 50 anggota dalam antrian, buku dikembalikan rata-rata setiap 3 hari sekali.

Dengan Queue biasa (FIFO), anggota ke-50 akan mendapat giliran setelah 49 anggota di depannya selesai.

➤ Estimasi waktu tunggu:

Waktu tunggu anggota ke-50 = 49 posisi $\times$ 3 hari/pengembalian = **147 hari**

➤ Perbandingan Biaya Operasi: Queue Biasa vs Priority Queue

Operasi	Queue Biasa	Priority Queue	Selisih (N=50)
enqueue	$O(1)$	$O(n)$ — cari posisi	50x lebih lambat
dequeue	$O(1)$	$O(1)$	Sama
peek	$O(1)$	$O(1)$	Sama

Untuk $N = 50$ anggota:

- Pada Queue biasa, anggota baru langsung ditambahkan di belakang antrian sehingga proses enqueue sangat cepat, yaitu $O(1)$.
- Pada Priority Queue, ketika anggota premium masuk, sistem harus mencari posisi yang tepat agar anggota premium berada di depan anggota biasa. Dalam worst-case, sistem bisa memeriksa hingga 50 posisi sehingga kompleksitasnya menjadi $O(n)$.

Artinya, Priority Queue memberikan keuntungan bagi anggota premium karena mereka bisa dilayani lebih cepat, tetapi proses penambahan anggota menjadi lebih lambat dibanding Queue biasa dan urutan pelayanan menjadi kurang adil bagi anggota reguler.

6.3. Pertanyaan 3

Graf Rekomendasi Ko-pinjam menggunakan BFS hop ≤ 2 . Jika setiap buku rata-rata terhubung ke 4 buku lain (degree=4) dan koleksi berisi 80 buku, estimasikan berapa node yang dijelajahi BFS dari satu sumber dalam 2 hop. Apakah semua rekomendasi relevan? Usulkan cara memfilter menggunakan bobot edge (frekuensi co-pinjam minimum) dan analisis dampaknya terhadap Big-O.

➤ Graf Rekomendasi — Estimasi Node BFS & Filter Bobot

Estimasi Node yang Dijelajahi BFS (≤ 2) Hop

Parameter: degree rata-rata = 4 (setiap buku terhubung ke ± 4 buku lain), koleksi = 80 buku.

- Hop 0 (1 sumber): jumlah node = 1.
- Hop 1 dari sumber: terhubung ke ± 4 buku lain langsung.
- Hop 2 dari sumber: setiap node hop-1 punya 4 tetangga, dan karena visited set mencegah node yang sama di kunjungi ulang maka, dikurangi dengan yang sudah dikunjungi. Maka estimasi node baru = $4 \times (4-1) = 12$ node baru.
- Total node dijelajahi BFS: 1 (sumber) + 4 (hop-1) + 12 (hop-2) = sekitar 17 node dari 80 total buku atau sekitar 21% dari seluruh koleksi.

Karena visited set mencegah kunjungan ganda, angka sebenarnya bisa lebih kecil jika ada overlap antar tetangga. Big-O BFS tetap $O(V+E)$ karena setiap node dan edge dikunjungi maksimal sekali.

➤ Relevansi Rekomendasi & Filter Bobot

Tidak semua rekomendasi relevan karena buku dengan bobot edge rendah (misalnya hanya dipinjam bersama 1 kali secara kebetulan) mungkin tidak benar-benar berhubungan secara konten. Implementasi menggunakan parameter `min_bobot` untuk menyaring:

- `min_bobot=1` (default): semua edge diikuti, rekomendasi paling banyak tapi bisa kurang relevan
- `min_bobot=2`: hanya edge yang frekuensinya ≥ 2 diikuti, rekomendasi lebih relevan
- `min_bobot=3`: filter ketat, rekomendasi paling relevan tapi paling sedikit

➤ Dampak terhadap Big-O

Dampak terhadap Big-O: penggunaan filter `min_bobot` membuat jumlah edge yang dijelajahi BFS menjadi lebih sedikit karena edge dengan hubungan lemah diabaikan. Misalnya, jika 30% edge memiliki bobot di bawah threshold `min_bobot=2`, maka BFS hanya memproses sekitar 70% edge yang ada. Akibatnya runtime menjadi lebih cepat dalam praktik, meskipun secara teori kompleksitas BFS tetap ditulis sebagai $O(V+E)$ karena struktur algoritmanya tidak berubah

6.4. Pertanyaan 4

Stack Riwayat Transaksi global menyimpan semua peminjaman/pengembalian. Jika perpustakaan memproses 200 transaksi/hari, berapa node Stack digunakan dalam satu bulan tanpa pembersihan? Usulkan strategi archiving: setelah N transaksi, pindahkan ke struktur lain (misalnya Linked List historis) agar Stack tetap ringkas. Analisis Big-O operasi archiving.

➤ Pertumbuhan Stack Tanpa Pembersihan

Parameter: 200 transaksi/hari, satu bulan (30 hari):

Total node Stack = $200 \times 30 = \mathbf{6.000 \text{ node}}$

Setiap node menyimpan dictionary Python sekitar 200–300 bytes dan Total memori = 1,2–1,8 MB. Meski relatif kecil untuk komputer modern, Stack yang tidak dibersihkan akan terus tumbuh: dalam satu semester (6 bulan) menjadi 36.000 node, dalam satu tahun 72.000 node. Selain itu, fungsi `tampilkan_stack()` yang $O(n)$ akan semakin lambat seiring waktu.

➤ Strategi Archiving

Implementasi menyediakan fungsi `ManajerRiwayat.archiving(maks_simpan=100)`: setelah Stack melebihi `maks_simpan` elemen, elemen-elemen lama (terbawah = tertua) dipindahkan ke Linked List historis dan dikembalikan ke pemanggil untuk disimpan ke file atau database. Stack hanya menyisakan `maks_simpan` transaksi terbaru yang relevan untuk operasi UNDO sehari-hari.

Tahap Archiving	Operasi	Big-O	Keterangan
1. Ambil semua elemen	tampilkan_stack()	$O(n)$	Traversal seluruh Stack
2. Pisah: simpan vs arsip	slicing list	$O(n)$	n = ukuran Stack saat ini
3. Rebuild Stack (baru)	push $\times$ maks_simpan	$O(\text{maks_simpan})$	Bangun ulang dari nol
4. Kembalikan arsip	return list	$O(1)$	Pointer ke list arsip

Total Big-O archiving: $O(n)$. Dengan maks_simpan = 100, archiving cukup dijalankan sekali setiap 100 transaksi. Amortized cost per transaksi: $O(n/100) = O(1)$ secara amortized.

6.5. Pertanyaan 5

Sistem ini menggunakan BST untuk katalog dan Graf untuk rekomendasi. Jelaskan bagaimana menambahkan fitur PENCARIAN MULTI-KRITERIA (misalnya: buku kategori 'Teknik' dengan status TERSEDIA, diurutkan berdasarkan judul). Apakah BST cocok untuk query ini? Usulkan penambahan struktur data (misalnya secondary index) dan analisis kompleksitasnya.

➤ Pencarian Multi-Kriteria & Secondary Index

Kemampuan BST untuk Multi-Kriteria Query

Untuk query multi kriteria, bst kurang cocok karena BST hanya terurut berdasarkan ISBN, bukan berdasarkan kategori atau status. Akibatnya sistem harus traversal seluruh BST, lalu memeriksa kategori setiap buku, lalu memeriksa status buku, dan menyortir hasil berdasarkan judul. Kompleksitas tanpa secondary Index:

- Ambil semua data dari BST dengan inorder traversal: $O(n)$
- Filter berdasarkan kategori dan status: $O(n)$
- Sorting berdasarkan judul: $O(n \log n)$

Sehingga total kompleksitas: $O(n + n + n \log n)$

Disederhanakan menjadi: $O(n \log n)$

Total tanpa secondary index: $O(n \log n)$ — tidak efisien jika query sering dilakukan

➤ Solusi: Secondary Index

Tambahkan secondary index berupa dictionary yang dibangun saat startup:

```
index_kategori = { 'Teknik': ['ISBN-0003', 'ISBN-0007', ...], 'Sains': [...], ... }
```

Kompleksitas dengan secondary index:

- Lookup kategori 'Teknik': $O(1)$ — langsung dari dictionary
- Untuk setiap ISBN di list: $BST.search()$ $O(\log n)$ untuk ambil status
- Filter status TERSEDIA: $O(k \log n)$ dengan k = jumlah buku Teknik, karena setiap pencarian di BST membutuhkan: $O(\log n)$
- Sorting berdasarkan judul: $O(k \log k)$

Total dengan secondary index:

$O(k \log n + k \log k)$, karena maka query menjadi jauh lebih efisien dibanding traversal seluruh BST.

Pendekatan	Kompleksitas Query	Kompleksitas Ruang Tambahan	Cocok Untuk
BST saja (tanpa index)	$O(n \log n)$	$O(1)$	Query jarang, memori terbatas
Secondary index (dict)	$O(k \log k)$, $k \ll n$	$O(n)$ untuk index	Query sering, memori tersedia

BAB VII

KESIMPULAN

7.1. RINGKASAN PENCAPAIAN

Proyek Smart Library Management & Recommendation System berhasil mengimplementasikan enam struktur data utama dalam satu sistem terintegrasi berbasis CLI:

1. Queue berbasis Linked List berhasil mengimplementasikan antrian pemesanan buku per-ISBN dengan enqueue/dequeue $O(1)$. Eksperimen mengkonfirmasi pertumbuhan waktu linear terhadap N .
2. Stack berbasis Linked List berhasil mengimplementasikan riwayat transaksi global dengan fitur undo (BATALKAN_TERAKHIR) $O(1)$. Dilengkapi metode archiving untuk menjaga ukuran Stack.
3. BST dengan kunci ISBN berhasil menyediakan pencarian $O(\log n)$ rata-rata. Eksperimen membuktikan tinggi pohon acak $= 12 \approx \log_2 80$, sedangkan pohon terurut $= 80 = n$ (worst-case).
4. Graf tak-berarah berbobot dengan BFS berhasil merekomendasikan buku berdasarkan pola ko-pinjam dalam $O(V+E)$. Pembatasan $\text{max_hop}=2$ efektif mempercepat BFS pada graf besar.
5. Shell Sort ($\sim O(n^{1.5})$) dan Merge Sort ($O(n \log n)$) berhasil diimplementasikan murni pada Linked List tanpa konversi ke Python list, menghasilkan laporan bulanan terurut.
6. CLI interaktif berhasil menyatukan semua fitur dalam 12 perintah dengan tampilan Big-O setiap operasi, seed reproduibel, dan startup $O(n \log n)$.

7.2. REFLEKSI TRADE-OFF DESAIN

- BST vs Hash Table: BST dipilih atas hash table untuk mendukung operasi inorder (katalog terurut ISBN) yang hash table tidak mendukung secara efisien. Biayanya adalah $O(\log n)$ rata-rata vs $O(1)$ hash table untuk search.
- Shell Sort pada LL vs Python sorted(): Shell Sort diimplementasikan langsung pada Linked List sesuai persyaratan, namun memerlukan array pointer sementara $O(n)$ karena LL tidak mendukung random access. Python sorted() akan 5–10× lebih cepat tetapi tidak memenuhi persyaratan implementasi dari nol.
- Queue Linked List vs deque: Queue Linked List memberikan $O(1)$ enqueue dan dequeue tanpa reallokasi memori, namun dengan overhead pointer per node. Python deque lebih cache-friendly untuk data kecil.

7.3. SARAN PENGEMBANGAN

Saran untuk pengembangan sistem ke depan:

1. Ganti BST standar dengan AVL Tree atau Red-Black Tree untuk menjamin $O(\log n)$ bahkan pada data terurut.
2. Tambahkan persistensi data menggunakan SQLite atau file JSON agar data tidak hilang saat program ditutup
3. Implementasikan antarmuka GUI atau REST API untuk menggantikan CLI, memudahkan penggunaan bagi petugas non-teknis.
4. Gunakan weighted BFS atau Dijkstra untuk rekomendasi yang mempertimbangkan bobot edge lebih akurat.
5. Tambahkan BST Range Query untuk fitur pencarian buku berdasarkan rentang ISBN atau kategori.

LAMP.

LAMPIRAN

LAMP.1. PEMBAGIAN TUGAS ANGGOTA

No.	Anggota (NIM)	Kontribusi
1	Fauzi Fajar Wijaya 25051030095	Implementasi / (queue, test_queue, modul_1) + README, AI_Log, Laporan, PPT
2	Muhammad Daffaa Adhirajasa 25051030086	Implementasi / (stack, test_stack, modul_2) + benchmark.py + README, AI_Log, Laporan, PPT
3	Itqon El Fikry Fausta 25051030083	Implementasi / (bst, test_bst, modul_3, modul_6) + data_model.py, generate_data.py, main.py, CLI + README, AI_Log, Laporan, PPT
4	Danar Hafidz A'isy 25051030112	Implementasi / (graph, test_graph, modul_4) + README, AI_Log, Laporan, PPT
5	Anargya Keano Arizona Marici 25051030096	Implementasi / (sorting, test_sorting, modul_5) + README, AI_Log, Laporan, PPT

LAMP.2. TABEL RUNTIME LENGKAP

QUEUE – enqueue & dequeue (rata-rata 5 ulangan)			
N	enqueue N (s)	dequeue N (s)	Big-O per operasi
20	0.000014	0.000018	O(1) per op
80	0.000035	0.000060	O(1) per op
300	0.000134	0.000218	O(1) per op

lampiran 1 Queue

STACK – push & pop (rata-rata 5 ulangan)			
N	push N (s)	pop N (s)	Big-O per operasi
20	0.000019	0.000016	O(1) per op
80	0.000034	0.000049	O(1) per op
300	0.000120	0.000206	O(1) per op

lampiran 2 Stack

BST – insert / search / inorder (rata-rata 5 ulangan)					
N	insert acak (s)	inserturut (s)	search (s)	inorder (s)	tinggi acak tinggiurut
20	0.000104	0.000079	0.000001	0.000007	10 20
80	0.000298	0.001016	0.000002	0.000022	12 80
300	0.001084	0.021517	0.000005	0.000076	16 300

lampiran 3 BST

GRAPH – add_copinjam & BFS rekomendasi (rata-rata 5 ulangan)					
N buku	V	E	deg rata-rata	add_copinjam N*2 (s)	BFS O(V+E) (s)
20	20	33	3.30	0.000092	0.000024
80	78	155	3.97	0.000329	0.000040
300	293	598	4.08	0.001697	0.000122

lampiran 4 Graph

SORTING – Shell Sort vs Merge Sort pada Linked List (rata-rata 5 ulangan)				
N	Shell Sort (s)	Merge Sort (s)	Big-O Shell	Big-O Merge
20	0.000844	0.000070	$\sim O(n^{1.5})$	$O(n \log n)$
80	0.001271	0.000299	$\sim O(n^{1.5})$	$O(n \log n)$
300	0.004370	0.000570	$\sim O(n^{1.5})$	$O(n \log n)$

lampiran 5 Shell sort vs merge sort

LAMP.3. LOG AI

1. =====
2. LOG PENGGUNAAN AI ASSISTANT – GABUNGAN SELURUH ANGGOTA
3. Proyek : Smart Library Management & Recommendation System
4. Mata Kuliah : ELT60213 Algoritma dan Struktur Data
5. Kelompok : [Nomor Kelompok]
6. Semester : 2 | TA 2025/2026
7. Tool AI : Claude Sonnet 4.6 (claude.ai) &
8. GPT-5.5 (chat.openai.com)
9. =====
- 10.
11. FORMAT LOG:
12. [Claude Sonnet 4.6] → review kode, analisis Big-O, debugging mendalam
13. [GPT-5.5] → pertanyaan konsep, penjelasan class/fungsi, error runtime
- 14.
15. =====
16. ANGGOTA 1 – ITQON EL FIKRY FAUSTA | Modul: BST Katalog Buku
17. =====
- 18.
19. [SESI 1 – Pemahaman Konsep BST]
- 20.
21. [Claude Sonnet 4.6]
22. PROMPT:
23. "Bisa jelaskan konsep Binary Search Tree secara singkat?
24. Terutama kenapa kuncinya harus unik dan bagaimana cara
25. kerjanya buat pencarian data."
- 26.

27. RESPONS (ringkasan):
 28. BST adalah pohon biner di mana setiap node punya kunci unik.
 29. **Node** kiri selalu lebih kecil dari root, node kanan selalu
 30. lebih besar. **Pencarian** efisien karena di setiap langkah,
 31. setengah dari pohon bisa diabaikan – sehingga $O(\log n)$
 32. untuk kasus rata-rata.
 33.
 34. -----
 35.
 36. [Claude Sonnet 4.6]
 37. PROMPT:
 38. "Kalau datanya dimasukkan dalam urutan terurut dari kecil
 39. ke besar, apa yang terjadi dengan bentuk pohonnya?"
 40.
 41. RESPONS (ringkasan):
 42. Jika data diinsert secara terurut, BST akan menjadi miring
 43. ke satu sisi (degenerate tree), sehingga tinggi pohon = n
 44. dan semua operasi menjadi $O(n)$ bukan $O(\log n)$.
 45. Ini disebut worst-case BST.
 46.
 47. -----
 48.
 49. [GPT-5.5]
 50. TANYA:
 51. "class BSTNode itu sebenarnya buat apa ya? apa bedanya
 52. sama class BSTKatalog?"
 53.
 54. JAWAB (ringkasan):
 55. **BSTNode** adalah blueprint satu node dalam pohon – menyimpan
 56. data buku dan pointer ke anak kiri/kanan. **BSTKatalog** adalah
 57. manajer pohon secara keseluruhan yang menyimpan root dan
 58. menyediakan operasi insert, search, delete. **Analogi:**
 59. **BSTNode** = satu kotak, **BSTKatalog** = rak yang mengelola
 60. susunan semua kotak.
 61.
 62. -----
 63.
 64. [SESI 2 – Implementasi BST]
 65.
 66. [Claude Sonnet 4.6]
 67. PROMPT:
 68. "Aku mau implementasi BST untuk katalog buku. Kunci yang
 69. dipakai adalah ISBN dalam format string. Gimana cara insert
 70. rekursif yang benar? Dan bagaimana cara tracking jumlah
 71. node tanpa pakai len() bawaan Python?"
 72.
 73. RESPONS (ringkasan):
 74. **Insert** rekursif membandingkan ISBN secara leksikografis
 75. di setiap node. Untuk tracking jumlah node, digunakan
 76. counter `_jumlah` manual yang hanya naik saat node benar-benar
 77. baru dibuat (bukan saat update duplikat).
 78.
 79. -----
 80.
 81. [GPT-5.5]
 82. TANYA:
 83. "fungsi `_insert_rekursif` itu dipanggil dari mana? kenapa
 84. ada underscore di depan namanya?"
 85.
 86. JAWAB (ringkasan):
 87. `_insert_rekursif` dipanggil dari dalam fungsi `insert()` –
 88. ini pola helper **private**. **Underscore** di depan nama fungsi
 89. adalah konvensi **Python** yang menandakan fungsi tersebut
 90. untuk pemakaian **internal** saja, tidak dimaksudkan dipanggil

```

91. dari luar class secara langsung.
92.
93. -----
94.
95. [Claude Sonnet 4.6]
96. PROMPT:
97. "Gimana cara implementasi delete BST yang benar? Aku bingung
98. handle kasus node dengan dua anak."
99.
100. RESPON (ringkasan):
101. Tiga kasus delete BST:
102. 1. Node daun: langsung hapus
103. 2. Satu anak: ganti node dengan anaknya
104. 3. Dua anak: cari inorder successor (node paling kiri
105.   di subtree kanan), salin datanya ke node yang dihapus,
106.   lalu hapus inorder successor dari subtree kanan.
107.
108. -----
109.
110. [GPT-5.5]
111. TANYA:
112. "fungsi _cari_minimum itu gunanya apa di dalam delete BST?"
113.
114. JAWAB (ringkasan):
115. _cari_minimum dipakai untuk menemukan inorder successor,
116. yaitu node dengan nilai terkecil di subtree kanan node yang
117. akan dihapus. Node ini dipakai sebagai pengganti node yang
118. dihapus ketika node tersebut punya dua anak, agar urutan
119. BST tetap terjaga setelah penghapusan.
120.
121. -----
122.
123. [Claude Sonnet 4.6]
124. PROMPT:
125. "Kenapa inorder traversal BST menghasilkan data terurut?
126. Bisa jelasin urutannya?"
127.
128. RESPON (ringkasan):
129. Inorder mengunjungi node dengan urutan kiri -> root -> kanan.
130. Karena semua node kiri lebih kecil dari root dan semua node
131. kanan lebih besar, urutan kunjungan secara alami menghasilkan
132. data dari terkecil ke terbesar.
133.
134. -----
135.
136. [SESI 3 – Analisis Big-O BST]
137.
138. [Claude Sonnet 4.6]
139. PROMPT:
140. "Kalau 80 buku disisipkan dengan urutan ISBN-0001 sampai
141. ISBN-0080 secara berurutan, berapa tinggi tree yang
142. terbentuk? Bandingkan dengan urutan acak."
143.
144. RESPON (ringkasan):
145. Insert berurutan menghasilkan pohon linear setinggi  $n=80$ 
146. (worst-case  $O(n)$ ). Insert acak menghasilkan pohon dengan
147. tinggi mendekati  $\log_2(80)$  sekitar 6-7 level. Ini dasar
148. Pertanyaan Analisis no.1 di laporan.
149.
150. -----
151.
152. [Claude Sonnet 4.6]
153. PROMPT:
154. "Kapan BST tetap lebih unggul dibanding Hash Table meski

```

```

155. Hash Table  $O(1)$  average untuk search?"
156.
157. RESPONS (ringkasan):
158. BST unggul untuk: range query, data terurut otomatis via
159. inorder, operasi predecessor/successor, dan tidak ada
160. collision handling. Hash Table unggul hanya untuk point
161. query  $O(1)$  average.
162.
163. -----
164.
165. [SESI 4 – Debugging & Pertanyaan Lanjutan BST]
166.
167. [GPT-5.5]
168. TANYA:
169. "aku dapat error ini waktu jalanin test BST:
170. AttributeError: 'NoneType' object has no attribute 'isbn'
171. kira-kira di bagian mana ya?"
172.
173. JAWAB (ringkasan):
174. Error ini terjadi karena fungsi search mengembalikan None
175. (buku tidak ditemukan), tapi kode pemanggil langsung akses
176. atributnya tanpa cek dulu. Fix: tambahkan pengecekan
177. 'if buku is None: return' sebelum mengakses buku.isbn
178. atau atribut lainnya.
179.
180. -----
181.
182. [Claude Sonnet 4.6]
183. PROMPT:
184. "Aku dapat error RecursionError: maximum recursion depth
185. exceeded saat insert 300 buku berurutan. Kenapa bisa terjadi
186. dan gimana cara fixnya?"
187.
188. RESPONS (ringkasan):
189. Error terjadi karena insert 300 buku secara berurutan membuat
190. pohon miring setinggi 300 level. Setiap insert rekursif
191. memanggil stack frame baru sampai Python kehabisan stack
192. (default limit ~1000). Fix jangka pendek: sys.setrecursionlimit.
193. Fix jangka panjang: implementasi iteratif atau AVL Tree.
194.
195. -----
196.
197. [Claude Sonnet 4.6]
198. PROMPT:
199. "Waktu aku coba delete root dari BST yang hanya punya satu
200. node, hasilnya aneh. Kodinganku jadi error. Ini kenapa?"
201.
202. RESPONS (ringkasan):
203. Saat delete root dengan satu node, fungsi mengembalikan None
204. sebagai node baru. Tapi kalau pemanggil tidak mengupdate
205. self._root = hasil, self._root masih menunjuk ke node lama.
206. Fix: pastikan hasil return dari _delete_rekursif selalu
207. di-assign kembali ke self._root.
208.
209. -----
210.
211. [Claude Sonnet 4.6]
212. PROMPT:
213. "Aku masih bingung soal kompleksitas ruang BST. Katanya
214.  $O(\log n)$  untuk call stack rekursif, tapi worst-case  $O(n)$ .
215. Kapan yang mana yang berlaku?"
216.
217. RESPONS (ringkasan):
218. Kompleksitas ruang call stack mengikuti tinggi pohon.

```

```

219. Pohon seimbang (insert acak) -> tinggi log n -> call stack  $O(\log n)$ .
220. Pohon miring (insert terurut) -> tinggi n -> call stack  $O(n)$ .
221. Worst-case ruang dan waktu keduanya  $O(n)$  pada kondisi sama.
222.
223. =====
224. ANGGOTA 2 – DANAR HAFIDZ A'ISY | Modul: Graph Rekomendasi Ko-pinjam
225. =====
226.
227. [SESI 1 – Pemahaman Konsep Graf]
228.
229. [Claude Sonnet 4.6]
230. PROMPT:
231. "Apa perbedaan representasi adjacency list vs adjacency
232. matrix untuk graf? Mana yang lebih cocok untuk sistem
233. rekomendasi buku?"
234.
235. RESPONS (ringkasan):
236. Adjacency matrix:  $O(V^2)$  ruang,  $O(1)$  cek edge, boros untuk
237. sparse graph. Adjacency list:  $O(V+E)$  ruang,  $O(\deg)$  cek edge,
238. efisien untuk sparse graph. Untuk rekomendasi buku (sparse),
239. adjacency list jauh lebih efisien.
240.
241. -----
242.
243. [GPT-5.5]
244. TANYA:
245. "self.adj di GraphRekBuku itu isinya apa ya formatnya
246. gimana?"
247.
248. JAWAB (ringkasan):
249. self.adj adalah dictionary dengan format:
250. { isbn: [(isbn_tetangga, bobot), ...] }
251. Contoh: {'ISBN-0001': [('ISBN-0002', 3), ('ISBN-0003', 1)]}
252. Artinya ISBN-0001 pernah dipinjam bersama ISBN-0002
253. sebanyak 3 kali dan bersama ISBN-0003 sebanyak 1 kali.
254.
255. -----
256.
257. [Claude Sonnet 4.6]
258. PROMPT:
259. "Gimana cara kerja BFS dan kenapa lebih cocok dari DFS
260. untuk rekomendasi dengan batasan hop?"
261.
262. RESPONS (ringkasan):
263. BFS menjelajah level per level sehingga mudah dibatasi
264. dengan max_hop. DFS pergi sejauh mungkin ke satu jalur
265. dulu sehingga sulit dikontrol batasannya. Untuk rekomendasi
266. dengan batasan jarak 2 hop, BFS adalah pilihan tepat.
267.
268. -----
269.
270. [SESI 2 – Implementasi Graph]
271.
272. [GPT-5.5]
273. TANYA:
274. "fungsi tambah_vertex di graph.py itu dipanggil kapan?
275. aku ga nemu di mana dia dipanggil secara eksplisit"
276.
277. JAWAB (ringkasan):
278. tambah_vertex dipanggil otomatis dari dalam add_copinjam.
279. Setiap kali dua buku dicatat sebagai ko-pinjam, keduanya
280. perlu terdaftar sebagai vertex dulu. Jadi tidak perlu
281. memanggil tambah_vertex secara manual dari luar – cukup
282. panggil add_copinjam dan vertex akan terdaftar sendiri

```

```

283. jika belum ada.
284.
285. -----
286.
287. [Claude Sonnet 4.6]
288. PROMPT:
289. "Aku mau implementasi BFS untuk rekomendasi buku, tapi
290. harus pakai Queue Linked List buatan sendiri, bukan
291. collections.deque. Gimana caranya?"
292.
293. RESPONS (ringkasan):
294. Queue Linked List diimport dari queue_ll.py. BFS standar
295. menggunakan queue untuk menyimpan node yang akan dikunjungi,
296. visited set untuk mencegah kunjungan ganda, dan counter
297. hop untuk membatasi kedalaman pencarian.
298.
299. -----
300.
301. [Claude Sonnet 4.6]
302. PROMPT:
303. "Bagaimana cara update bobot edge jika dua buku yang sama
304. dipinjam bersama lagi? Apakah perlu buat edge baru?"
305.
306. RESPONS (ringkasan):
307. Tidak perlu buat edge baru. Scan adjacency list node tersebut
308. ( $O(\text{deg})$ ), temukan entri pasangannya, dan naikan bobotnya +1.
309. Jika belum ada, baru tambahkan entry baru dengan bobot 1.
310.
311. -----
312.
313. [SESI 3 – Analisis Big-O Graph]
314.
315. [Claude Sonnet 4.6]
316. PROMPT:
317. "Kalau setiap buku rata-rata terhubung ke 4 buku lain
318. dan koleksi ada 80 buku, berapa node yang dijelajahi
319. BFS dari satu sumber dalam 2 hop?"
320.
321. RESPONS (ringkasan):
322. Estimasi: hop 1 = 4 node, hop 2 =  $4 \times 4 = 16$  node.
323. Total sekitar 20 node dari 80 (25%). Karena visited set
324. mencegah kunjungan ganda, angka bisa lebih kecil jika ada
325. overlap. Big-O tetap  $O(V+E)$ .
326.
327. -----
328.
329. [SESI 4 – Debugging & Pertanyaan Lanjutan Graph]
330.
331. [GPT-5.5]
332. TANYA:
333. "waktu aku jalanin REKOMENDASI di CLI dapat error:
334. TypeError: cannot unpack non-iterable NoneType object
335. ini di bagian mana ya?"
336.
337. JAWAB (ringkasan):
338. Error ini muncul saat hasil rekomendasikan() mengembalikan
339. None alih-alih list kosong []. Cek bagian return di fungsi
340. rekomendasikan – jika isbn tidak ada di graf, pastikan
341. return [] bukan return None. CLI mencoba unpack tuple
342. (isbn, bobot, hop) dari hasil, kalau None maka unpack gagal.
343.
344. -----
345.
346. [Claude Sonnet 4.6]

```

```

347. PROMPT:
348. "BFS ku masuk infinite loop kalau ada siklus di graf.
349. Misalnya A-B, B-C, C-A. Gimana cara fixnya?"
350.
351. RESPONS (ringkasan):
352. Fix: gunakan visited set dan tambahkan node ke visited
353. SEBELUM dimasukkan ke queue, bukan setelah dikeluarkan.
354. Dengan begitu, saat C menemukan A lagi, A sudah ada di
355. visited dan langsung diskip.
356.
357. -----
358.
359. [Claude Sonnet 4.6]
360. PROMPT:
361. "Gimana cara representasi bobot edge di adjacency list?
362. Aku bingung struktur datanya harus seperti apa."
363.
364. RESPONS (ringkasan):
365. Simpan adjacency list sebagai dict: isbn -> list of tuple
366. (isbn_tetangga, bobot). Saat add_copinjam, scan list tuple
367. untuk cari pasangan – jika ketemu naikan bobot, jika tidak
368. append tuple baru dengan bobot 1.
369.
370. =====
371. ANGGOTA 3 & 4 – [Nama] | Modul: Queue & Stack
372. =====
373.
374. [SESI 1 – Pemahaman Queue dan Stack]
375.
376. [GPT-5.5]
377. TANYA:
378. "class Queue di queue.py itu buat apa bedanya sama
379. list biasa Python?"
380.
381. JAWAB (ringkasan):
382. Queue ini mengimplementasikan antrian FIFO berbasis Linked
383. List dari nol tanpa pakai list Python. Bedanya: Queue ini
384. tidak bisa diakses by index, hanya bisa enqueue (tambah
385. belakang) dan dequeue (ambil depan), keduanya O(1). List
386. Python bisa diakses random tapi insert/delete di depan O(n)
387. karena harus geser semua elemen.
388.
389. -----
390.
391. [Claude Sonnet 4.6]
392. PROMPT:
393. "Apa perbedaan mendasar Queue dan Stack? Dan kenapa keduanya
394. lebih baik diimplementasi dengan Linked List daripada array
395. untuk kasus perpustakaan ini?"
396.
397. RESPONS (ringkasan):
398. Queue = FIFO, cocok untuk antrian. Stack = LIFO, cocok
399. untuk undo. Linked List lebih baik karena tidak perlu
400. pre-allocate ukuran, insert/delete di ujung selalu O(1)
401. tanpa shift elemen, dan ukuran dinamis.
402.
403. -----
404.
405. [Claude Sonnet 4.6]
406. PROMPT:
407. "Kenapa Queue perlu pointer tail selain pointer head?
408. Apa yang terjadi kalau tidak ada pointer tail?"
409.
410. RESPONS (ringkasan):

```

```

411. Tanpa tail pointer, setiap enqueue harus traversal dari
412. head ke node terakhir =  $O(n)$ . Dengan tail pointer,
413. enqueue langsung sambung ke tail =  $O(1)$ .
414.
415. -----
416.
417. [SESI 2 – Implementasi Queue]
418.
419. [GPT-5.5]
420. TANYA:
421. "_head dan _tail di Queue itu nunjuk ke apa pas Queue
422. baru dibuat dan pas Queue kosong setelah di-dequeue semua?"
423.
424. JAWAB (ringkasan):
425. Saat Queue baru dibuat: _head = None, _tail = None.
426. Saat ada elemen: _head nunjuk ke node terdepan (pertama
427. keluar), _tail nunjuk ke node paling belakang (terakhir
428. masuk). Saat Queue jadi kosong lagi setelah semua
429. di-dequeue: keduanya harus di-reset ke None. Kalau _tail
430. tidak di-reset, enqueue berikutnya akan nyambung ke node
431. lama yang sudah tidak valid.
432.
433. -----
434.
435. [Claude Sonnet 4.6]
436. PROMPT:
437. "Ada bug di Queue ku. Setelah dequeue semua elemen lalu
438. enqueue lagi, program error. Kira-kira masalahnya di mana?"
439.
440. RESPON (ringkasan):
441. Pointer tail tidak direset ke None saat queue jadi kosong.
442. Fix: tambahkan 'if self._head is None: self._tail = None'
443. setelah menggeser head di dalam dequeue.
444.
445. -----
446.
447. [Claude Sonnet 4.6]
448. PROMPT:
449. "Bagaimana cara batalkan pesanan anggota dari tengah antrian?
450. Kan Queue tidak support random delete."
451.
452. RESPON (ringkasan):
453. Rebuild: ambil semua elemen ke list sementara  $O(k)$ , filter
454. elemen yang dibatalkan, lalu bangun ulang Queue baru tanpa
455. elemen tersebut. Big-O:  $O(k)$ .
456.
457. -----
458.
459. [SESI 3 – Implementasi Stack]
460.
461. [GPT-5.5]
462. TANYA:
463. "self._top di Stack itu selalu nunjuk ke node yang mana?
464. apa berubah setiap kali push dan pop?"
465.
466. JAWAB (ringkasan):
467. Ya, _top selalu berubah. Saat push, node baru dibuat dan
468. _top digeser ke node baru tersebut. Saat pop, _top digeser
469. ke node di bawahnya. Jadi _top selalu menunjuk ke node
470. paling atas tumpukan – node yang paling terakhir di-push
471. dan akan paling pertama di-pop.
472.
473. -----
474.

```



```

475. [Claude Sonnet 4.6]
476. PROMPT:
477. "Gimana cara implementasi fitur UNDO yang benar menggunakan
478. Stack? Apa saja yang perlu disimpan di tiap node Stack?"
479.
480. RESPONS (ringkasan):
481. Dict di Stack berisi: tx_id, aksi, nim, isbn, waktu, durasi.
482. Saat UNDO: PINJAM -> kembalikan status BST ke TERSEDIA,
483. KEMBALIKAN -> kembalikan status BST ke DIPINJAM,
484. PESAN -> hapus nim dari antrian ISBN tersebut.
485.
486. -----
487.
488. [Claude Sonnet 4.6]
489. PROMPT:
490. "Kalau perpustakaan memproses 200 transaksi per hari, dalam
491. sebulan Stack bisa sangat besar. Gimana cara handle ini?"
492.
493. RESPONS (ringkasan):
494. Strategi archiving: setelah Stack mencapai batas N elemen,
495. N elemen terlama dipindah ke struktur lain. Stack hanya
496. menyimpan transaksi terbaru yang relevan untuk UNDO.
497. Big-O archiving: O(n) traversal seluruh Stack sekali.
498.
499. -----
500.
501. [SESI 4 – Debugging & Pertanyaan Lanjutan Queue & Stack]
502.
503. [GPT-5.5]
504. TANYA:
505. "aku dapat error waktu jalanin BATALKAN_TERAKHIR:
506. KeyError: 'aksi'
507. ini maksudnya apa?"
508.
509. JAWAB (ringkasan):
510. KeyError: 'aksi' berarti dict transaksi yang di-pop dari
511. Stack tidak punya key 'aksi'. Kemungkinan: saat catat()
512. di ManajerRiwayat membentuk dict transaksi, key 'aksi'
513. tidak ikut dimasukkan atau nama key-nya tidak konsisten
514. (misal 'action' bukan 'aksi'). Cek format dict di fungsi
515. catat() dan pastikan konsisten dengan yang diakses di
516. handler BATALKAN_TERAKHIR.
517.
518. -----
519.
520. [Claude Sonnet 4.6]
521. PROMPT:
522. "Aku bingung kenapa peek() di Stack ku mengembalikan None
523. padahal Stack tidak kosong. Aku sudah push beberapa elemen."
524.
525. RESPONS (ringkasan):
526. Kemungkinan besar self._top tidak diupdate dengan benar
527. saat push. Cek apakah baru.next = self._top diletakkan
528. SEBELUM self._top = baru. Jika urutan terbalik, node baru
529. justru menunjuk ke dirinya sendiri dan peek tidak benar.
530.
531. -----
532.
533. [Claude Sonnet 4.6]
534. PROMPT:
535. "Fungsi tampilkan_stack() ku terasa lambat kalau Stack
536. sudah besar. Ini wajar atau ada yang salah?"
537.
538. RESPONS (ringkasan):

```

```

539. Wajar, karena tampilkan_stack() harus traversal dari top
540. ke node terbawah =  $O(n)$ . Ini trade-off Linked List –
541. akses random  $O(n)$ . Yang penting push/pop tetap  $O(1)$ .
542.
543. =====
544. ANGGOTA 4 – ANARGYA KEANO ARIZONA MARICI | Modul: Sorting (Shell Sort & Merge Sort)
545. =====
546.
547. [SESI 1 – Pemahaman Algoritma Sorting]
548.
549. [GPT-5.5]
550. TANYA:
551. "sorting.py itu dipanggil dari modul mana? dan fungsinya
552. buat apa di sistem perpustakaan ini?"
553.
554. JAWAB (ringkasan):
555. sorting.py diimport oleh modul_5.py (ManajerLaporan).
556. Fungsinya untuk menyortir data laporan bulanan:
557. shell_sort_durasi mengurutkan riwayat peminjaman berdasarkan
558. durasi (terlama di atas) dan merge_sort_frekuensi
559. mengurutkan frekuensi peminjaman per ISBN (terbanyak di atas).
560. Hasilnya ditampilkan saat perintah LAPORAN_BULAN di CLI.
561.
562. -----
563.
564. [Claude Sonnet 4.6]
565. PROMPT:
566. "Apa perbedaan Shell Sort dan Merge Sort dari segi Big-O
567. dan kapan masing-masing lebih unggul?"
568.
569. RESPONS (ringkasan):
570. Shell Sort:  $\sim O(n^{1.5})$  dengan gap sequence Knuth, in-place,
571. lebih cepat untuk data kecil atau hampir terurut.
572. Merge Sort:  $O(n \log n)$  guaranteed, stable sort, butuh
573. memori tambahan  $O(n)$ . Untuk laporan bulanan, keduanya
574. bisa dibandingkan runtimenya secara empiris.
575.
576. -----
577.
578. [Claude Sonnet 4.6]
579. PROMPT:
580. "Kenapa Shell Sort lebih baik dari Bubble Sort atau
581. Insertion Sort biasa untuk kasus ini?"
582.
583. RESPONS (ringkasan):
584. Bubble Sort dan Insertion Sort keduanya  $O(n^2)$  worst-case.
585. Shell Sort memperbaiki ini dengan membandingkan elemen
586. yang jauh terlebih dahulu. Gap sequence Knuth
587. (1, 4, 13, 40...) memberikan kompleksitas  $\sim O(n^{1.5})$ .
588.
589. -----
590.
591. [SESI 2 – Implementasi Sorting pada Linked List]
592.
593. [GPT-5.5]
594. TANYA:
595. "nodes = [] di shell_sort_durasi itu buat apa? kenapa
596. perlu dikumpulin dulu ke array?"
597.
598. JAWAB (ringkasan):
599. nodes = [] mengumpulkan semua pointer node Linked List
600. ke array sementara. Ini diperlukan karena Shell Sort butuh
601. akses acak ke elemen ke-i dan ke-(i+gap), sedangkan Linked
602. List tidak punya random access  $O(1)$ . Dengan pointer di array,

```

```

603. akses O(1) bisa dilakukan. Swap dilakukan pada data di dalam
604. node bukan pointer-nya, sehingga struktur Linked List tetap.
605.
606. -----
607.
608. [Claude Sonnet 4.6]
609. PROMPT:
610. "Bagaimana cara implementasi Shell Sort pada Linked List?
611. Kan Linked List tidak punya random access seperti array."
612.
613. RESPON (ringkasan):
614. Kumpulkan semua pointer node ke array temporary O(n),
615. lakukan Shell Sort pada array dengan swap data (bukan pointer),
616. hasilnya otomatis mencerminkan urutan di Linked List.
617. Struktur pointer Linked List tidak diubah sama sekali.
618.
619. -----
620.
621. [GPT-5.5]
622. TANYA:
623. "dummy = LLNode di _merge_descending itu gunanya apa?
624. kenapa harus pakai node dummy?"
625.
626. JAWAB (ringkasan):
627. Node dummy (sentinel) dipakai sebagai titik awal hasil merge
628. sementara. Tanpanya, kita perlu if-else khusus untuk
629. menentukan head hasil merge. Dengan dummy, kita selalu
630. sambungkan ke dummy.next dan di akhir return dummy.next
631. untuk melewati dummy itu sendiri. Teknik umum untuk
632. menyederhanakan logika merge pada Linked List.
633.
634. -----
635.
636. [Claude Sonnet 4.6]
637. PROMPT:
638. "Gimana cara implementasi Merge Sort pada Linked List
639. tanpa mengconvert ke array Python dulu?"
640.
641. RESPON (ringkasan):
642. Murni dengan pointer manipulation:
643. 1. Split: slow/fast pointer cari tengah lalu putus jadi dua
644. 2. Rekursif sort dua bagian
645. 3. Merge: gabung dua list terurut dengan sentinel node
646. Tidak perlu convert ke array sama sekali.
647.
648. -----
649.
650. [SESI 3 – Debugging & Pertanyaan Lanjutan Sorting]
651.
652. [GPT-5.5]
653. TANYA:
654. "waktu jalanin test sorting dapat error:
655. AssertionError: assert [14, 14, 7] == [21, 14, 7]
656. berarti hasil sortingnya salah ya? bisa kenapa?"
657.
658. JAWAB (ringkasan):
659. Ya, urutan descending tidak benar. Kemungkinan di Shell Sort:
660. kondisi perbandingan salah arah. Cek baris while di Shell Sort:
661. seharusnya 'nodes[j-gap].data.get(durasi, 0) < temp_data'
662. (tanda lebih kecil untuk descending). Kalau tanda-nya dibalik
663. menjadi > maka hasilnya ascending bukan descending.
664.
665. -----
666.

```

```

667. [Claude Sonnet 4.6]
668. PROMPT:
669. "Merge Sort ku kadang hasilnya benar, kadang salah. Tidak
670. konsisten. Aku sudah cek logika merge-nya tapi kelihatannya
671. bener. Ini bisa disebabkan apa?"
672.
673. RESPONS (ringkasan):
674. Kemungkinan masalah ada di _split_tengah. Jika slow.next
675. tidak di-set ke None setelah split, kedua bagian masih
676. terhubung. Pastikan slow.next = None setelah menemukan
677. tengah untuk benar-benar memutus dua bagian.
678.
679. -----
680.
681. [Claude Sonnet 4.6]
682. PROMPT:
683. "Kenapa hasil benchmark Shell Sort kadang lebih cepat dari
684. Merge Sort padahal Big-O Merge Sort lebih baik?"
685.
686. RESPONS (ringkasan):
687. Big-O mengabaikan konstanta. Shell Sort in-place tanpa
688. overhead rekursif. Untuk n kecil, konstanta Shell Sort
689. yang kecil bisa mengalahkan keunggulan asimptotik Merge Sort.
690. Ini wajar dan mendukung analisis di laporan.
691.
692. =====
693. ANGGOTA 5 – ITQON EL FIKRY FAUSTA | Modul: CLI & Integrasi
694. =====
695.
696. [SESI 1 – Perancangan CLI]
697.
698. [GPT-5.5]
699. TANYA:
700. "self._handler di CLI itu isinya apa dan cara kerjanya
701. gimana waktu user ketik perintah?"
702.
703. JAWAB (ringkasan):
704. self._handler adalah dictionary yang memetakan string
705. perintah ke fungsi handler-nya. Contoh:
706. {'CARI_BUKU': self._handle_cari_buku, 'PINJAM': ...}
707. Waktu user ketik perintah, CLI split input jadi token,
708. ambil token pertama sebagai perintah, lalu lookup di
709. self._handler. Kalau ketemu, panggil fungsi handler-nya.
710. Ini disebut dispatch table – lebih rapi dari if-elif panjang.
711.
712. -----
713.
714. [Claude Sonnet 4.6]
715. PROMPT:
716. "Bagaimana cara membuat CLI yang bersih untuk sistem
717. perpustakaan ini? Supaya setiap perintah bisa ditangani
718. secara modular tanpa banyak if-else bertumpuk."
719.
720. RESPONS (ringkasan):
721. Pola dispatch table: buat dict yang memetakan string
722. perintah ke method handler. Ini menghindari if-elif panjang,
723. mudah ditambah perintah baru, dan setiap handler bisa
724. ditest secara terpisah.
725.
726. -----
727.
728. [Claude Sonnet 4.6]
729. PROMPT:
730. "Gimana cara handle error di CLI supaya program tidak crash

```

```

731. kalau user salah input perintah atau argumen kurang?"
732.
733. RESPONS (ringkasan):
734. Gunakan try-except di level loop utama untuk menangkap
735. semua exception tanpa menghentikan program. Setiap handler
736. juga perlu validasi argumen sendiri dengan cek len(token).
737.
738. -----
739.
740. [SESI 2 – Integrasi Antar Modul]
741.
742. [GPT-5.5]
743. TANYA:
744. "di main.py ada cli.jalankan() – fungsi itu ada di mana
745. dan ngapain aja di dalamnya?"
746.
747. JAWAB (ringkasan):
748. jalankan() ada di class CLI di modul_6.py. Isinya adalah
749. loop utama program: cetak banner, lalu terus-menerus
750. tunggu input user dengan input('>>'), parse perintah,
751. cari handler di self._handler, dan panggil handler-nya.
752. Loop baru berhenti kalau user ketik KELUAR atau ada
753. KeyboardInterrupt (Ctrl+C).
754.
755. -----
756.
757. [Claude Sonnet 4.6]
758. PROMPT:
759. "Gimana cara koordinasi antara modul-modul yang ada saat
760. perintah KEMBALIKAN dijalankan? Banyak yang harus diupdate
761. sekaligus."
762.
763. RESPONS (ringkasan):
764. Urutan koordinasi KEMBALIKAN:
765. 1. ManajerKatalog.cari_buku -> cek buku ada dan DIPINJAM
766. 2. ManajerAntrian.proses_pengembalian -> dequeue pemesanan
767. 3. ManajerKatalog.update status -> TERSEDIA atau DIPESAN
768. 4. ManajerRiwayat.catat -> push ke Stack
769. 5. ManajerRekomendasi.catat_kembalikan -> update sesi graf
770.
771. -----
772.
773. [SESI 3 – Debugging & Fix]
774.
775. [GPT-5.5]
776. TANYA:
777. "dapat error ini waktu jalanin main.py:
778. 'ManajerRekomendasi' object has no attribute 'rekomendasikan'
779. padahal fungsinya udah aku bikin di modul_4.py"
780.
781. JAWAB (ringkasan):
782. Error ini berarti Python tidak menemukan method rekomendasi
783. di dalam class ManajerRekomendasi. Kemungkinan: indentasi
784. fungsi rekomendasi salah – tidak berada di dalam class
785. (harus diindentasi 4 spasi ke kanan dari batas class).
786. Cek modul_4.py dan pastikan def rekomendasi(self, ...)
787. ada di dalam blok class, bukan di level modul (rata kiri).
788.
789. -----
790.
791. [Claude Sonnet 4.6]
792. PROMPT:
793. "Ada error ModuleNotFoundError saat jalanin test.
794. generate_data.py menggunakan from src.data_model import Buku

```

```

795. tapi error pas dijalankan dari root project."
796.
797. RESPON (ringkasan):
798. Prefix src. tidak valid saat file dijalankan dari dalam src/
799. karena Python mencari modul src di dalam src. Fix: ganti jadi
800. from data_model import Buku karena src/ sudah ada di sys.path.
801.
802. -----
803.
804. [GPT-5.5]
805. TANYA:
806. "aku dapat error waktu pytest:
807. ImportError: cannot import name 'Queue' from
808. 'data_structures.queue'
809. padahal file queue_ll.py aku udah buat"
810.
811. JAWAB (ringkasan):
812. Masalahnya nama file yang diimport tidak cocok. Di test_queue.py
813. tertulis 'from data_structures.queue import Queue' tapi nama
814. file aslinya queue_ll.py bukan queue.py. Fix: ganti import
815. menjadi 'from data_structures.queue_ll import Queue' atau
816. rename file-nya – pilih salah satu dan konsisten di semua file.
817.
818. -----
819.
820. [Claude Sonnet 4.6]
821. PROMPT:
822. "Di benchmark ada bug aneh, hasil sorting selalu pakai
823. data dari iterasi terakhir loop, bukan data dari iterasi
824. yang sesuai."
825.
826. RESPON (ringkasan):
827. Ini closure bug klasik Python. Fungsi yang didefinisikan
828. di dalam loop menangkap variabel by reference bukan by value.
829. Fix: tambahkan default argument, contoh:
830. def uji_shell(tx_list=tx_list).
831.
832. -----
833.
834. [SESI 4 – Pertanyaan Lanjutan CLI & Integrasi]
835.
836. [Claude Sonnet 4.6]
837. PROMPT:
838. "Aku mau nampilin Big-O di setiap output CLI. Tapi aku
839. bingung cara passing info Big-O dari modul ke CLI."
840.
841. RESPON (ringkasan):
842. Cara paling bersih: setiap fungsi modul mengembalikan dict
843. yang sudah menyertakan key 'big_o'. CLI cukup print
844. hasil['big_o'] tanpa perlu tahu implementasi dalamnya.
845.
846. -----
847.
848. [Claude Sonnet 4.6]
849. PROMPT:
850. "Program aku crash dengan KeyError saat user ketik perintah
851. dengan huruf kecil, misalnya 'cari_buku' bukan 'CARI_BUKU'."
852.
853. RESPON (ringkasan):
854. Fix: tambahkan .upper() saat parse token perintah.
855. Ubah perintah = token[0] menjadi perintah = token[0].upper().
856.
857. -----
858.

```

859. [GPT-5.5]
860. TANYA:
861. "fungsi _cari_peminjam_aktif di CLI itu gunanya apa?
862. dipanggil dari handler mana?"
863.
864. JAWAB (ringkasan):
865. _cari_peminjam_aktif dipanggil dari _handle_kembalikan.
866. Fungsinya mencari NIM anggota yang sedang meminjam suatu
867. buku berdasarkan riwayat Stack – scan dari atas ke bawah,
868. cari transaksi PINJAM untuk ISBN tersebut yang belum di-undo.
869. Hasilnya dipakai untuk update sesi rekomendasi saat buku
870. dikembalikan, agar graf ko-pinjam anggota tersebut ikut
871. diperbarui.
872.
873. =====
874. ANGGOTA SEMUA – Analisis Hasil Benchmark vs Teori
875. =====
876.
877. [SESI BERSAMA – Diskusi Hasil Eksperimen Runtime]
878.
879. [Claude Sonnet 4.6]
880. PROMPT:
881. "Kami sudah jalanin benchmark.py dan dapat hasil tabel
882. runtime untuk N=20, N=80, N=300. Tapi bingung cara
883. bacanya dan gimana cara menghubungkan ke teori Big-O
884. yang sudah kami pelajari. Bisa bantu analisis?"
885.
886. RESPON (ringkasan):
887. Cara membaca tabel benchmark: lihat rasio waktu antar N.
888. $O(1)$ -> waktu hampir konstan. $O(\log n)$ -> naik sangat lambat.
889. $O(n)$ -> naik linear. $O(n^2)$ -> naik 4x saat N naik 2x.
890. Bandingkan rasio empiris dengan prediksi teori.
891.
892. -----
893.
894. [Claude Sonnet 4.6]
895. PROMPT:
896. "BST insert acak N=20=0.000089s, N=80=0.000412s,
897. N=300=0.001823s. Sedangkan insert terurut N=20=0.000095s,
898. N=80=0.000687s, N=300=0.004521s. Kenapa insert terurut
899. jauh lebih lambat di N=300?"
900.
901. RESPON (ringkasan):
902. Bukti empiris worst-case BST. Insert acak: rasio $300/20 \approx 15x$
903. untuk 15x lipat N -> mendekati $O(n \log n)$ total. Insert terurut:
904. rasio $\approx 47x$ -> mendekati $O(n^2)$ total. Gap semakin lebar
905. seiring N – persis prediksi teori $O(\log n)$ vs $O(n)$ per operasi.
906.
907. -----
908.
909. [GPT-5.5]
910. TANYA:
911. "kami bingung kenapa BFS graph hasilnya jauh lebih cepat
912. dari yang kami ekspektasikan. N=300 cuma 0.000312s.
913. apa ini normal?"
914.
915. JAWAB (ringkasan):
916. Normal, karena graf yang dihasilkan sparse – tidak semua 300
917. buku terhubung satu sama lain. E (jumlah edge) jauh lebih
918. kecil dari V^2 sehingga $O(V+E)$ mendekati $O(V)$ dalam kasus ini.
919. BFS terasa hampir linear terhadap V. Jika jumlah transaksi
920. ko-pinjam ditambah hingga setiap buku terhubung ke banyak
921. buku lain, barulah BFS terasa lebih lambat secara proporsional.
922.

```

923. -----
924.
925. [Claude Sonnet 4.6]
926. PROMPT:
927. "Hasil Shell Sort N=20=0.000021s, N=80=0.000142s,
928. N=300=0.001876s. Merge Sort N=20=0.000018s, N=80=0.000098s,
929. N=300=0.000934s. Kenapa Merge Sort lebih cepat di N=300
930. tapi hampir sama di N=20?"
931.
932. RESPONS (ringkasan):
933. Fenomena crossover point. Di N kecil, konstanta lebih dominan
934. dari Big-O – Shell Sort in-place tanpa overhead rekursif bisa
935. menang. Di N=300:  $300^{1.5} \approx 5196$  operasi vs  $300 \times \log_2(300) \approx 2430$ 
936. – Merge Sort 2x lebih efisien, cocok dengan rasio empiris.
937.
938. -----
939.
940. [GPT-5.5]
941. TANYA:
942. "gimana cara nulis hasil benchmark ini di laporan? apakah
943. cukup copy paste tabel runtimenya saja?"
944.
945. JAWAB (ringkasan):
946. Tidak cukup hanya tabel. Struktur analisis yang baik:
947. (1) tampilkan tabel runtime, (2) hitung rasio waktu antar N,
948. (3) bandingkan rasio empiris dengan prediksi teori Big-O,
949. (4) jelaskan jika ada deviasi dan kenapa bisa terjadi,
950. (5) tarik kesimpulan apakah implementasi terbukti efisien.
951. Hindari hanya menulis "hasilnya sesuai teori" – tunjukkan
952. angka rasionya secara eksplisit agar analisis terlihat kuat.
953.
954. =====
955. CATATAN PENUTUP
956. =====
957.
958. Seluruh output AI telah dibaca, dipahami, dan diverifikasi
959. oleh masing-masing anggota sebelum diintegrasikan ke proyek.
960. Setiap anggota tetap bertanggung jawab penuh untuk menjelaskan
961. modul yang dikerjakannya saat sesi presentasi dan tanya jawab.
962.
963. Penggunaan AI dibagi sebagai berikut:
964. - Claude Sonnet 4.6 : review kode mendalam, analisis Big-O,
965.                        debugging kompleks, integrasi antar modul
966. - GPT-5.5           : pertanyaan konsep, penjelasan class/fungsi,
967.                        error runtime, pemahaman alur program
968.
969. Screenshot percakapan tersedia di folder AI_Log/screenshots/
970.
971. =====
972. END OF LOG
973. =====
974.

```

```

=====

```